

INSTRUKCJA OBSŁUGI

Maprisma

& AI Agents Map

Twoi agenci AI, lokalne modele, serwery MCP i skille na jednej mapie

Maprisma 0.12.0 · AI Agents Map 0.26.1

listopad 2026

O tej instrukcji

Ta instrukcja opisuje jedną aplikację, która występuje w dwóch wydaniach:

Wydanie	Maprisma	AI Agents Map
Czym jest	Samodzielna aplikacja GTK 4 / libadwaita dla dowolnego pulpitu Linux	Wtyczka do pulpitu Omarchy, otwierana przyciskiem na pasku
Instalacja z	AUR (`maprisma-bin`) lub AppImage	Omarchy Plugin Marketplace
Wygląd	Podąża za jasnym/ciemnym trybem i kolorem akcentu systemu	Kolory motywu Omarchy lub wygląd Adwaita z GNOME
Folder ustawień	`~/.config/maprisma/`	`~/.config/omarchy/ai-agents-map/`

Wszystko inne — Konfiguracja, mapa, łączenie przez rysowanie, lokalne LLM, modele, serwery MCP i skille — jest w obu takie samo. To, co dotyczy tylko jednego wydania, jest oznaczone jako MAPRISMA lub OMARCHY.

Konwencje

Dodaj	Przycisk, karta lub pozycja menu w oknie.
Ustawienia › **Wygląd**	Ścieżka w interfejsie: otwórz ustawienia, potem **Wygląd**.
`~/.agents/skills`	Plik, folder lub polecenie. `~` to twój folder domowy.
`Ctrl`+`,`	Skrót klawiszowy.

UWAGA Uwagi podają przydatne tło.

WSKAZÓWKA Wskazówki pokazują szybszą drogę lub dobry nawyk.

OSTROŻNIE Ramki „Ostrożnie” wskazują coś, czego trudno cofnąć.

Spis treści

1 **Wprowadzenie** 1
- 1.1 Co robi 1
- 1.2 Podstawowe pojęcia 2
- 1.3 Nic się nie zmienia bez podglądu 2

2 **Instalacja** 3
- 2.1 Instalacja Maprisma 3
- 2.2 Pierwsze uruchomienie 3
- 2.3 Usuwanie aplikacji 4

3 **Okno w skrócie** 5
- 3.1 Konfiguracja 6
- 3.2 Wybór, co pokazuje mapa 8
- 3.3 Praca z mapą 9
- 3.4 Stan kolumn 10
- 3.5 Inteligentne karty na mapie 11

4 **Agenci** 13
- 4.1 Szczegóły agenta 13
- 4.2 Uruchamianie i zatrzymywanie agentów 14
- 4.3 Ruch sieciowy 15
- 4.4 Podagenci 15
- 4.5 Współpracujący agenci w podglądzie Flutter 16
- 4.6 Koszty w podglądzie Flutter 16
- 4.7 Szczegóły w podglądzie Flutter 17
- 4.8 Karty terminali, pole użycia i model startowy w podglądzie Flutter 17
- 4.9 Konfiguruj… 17
- 4.10 Logowanie 17
- 4.11 Klucze API w podglądzie Flutter 18
- 4.12 Aktualizowanie agentów 18
- 4.13 Aktualizacja działającego agenta w podglądzie Flutter 19
- 4.14 Agenci, których aplikacja może zainstalować 19
- 4.15 Nowi agenci 19

5 **Lokalne LLM** 20
- 5.1 Instalacja lokalnego LLM 20
- 5.2 Instalowanie w podglądzie Flutter 20
- 5.3 Uruchamianie i zatrzymywanie z agentami 21
- 5.4 Szczegóły lokalnego LLM 21
- 5.5 Wydajność 21
- 5.6 Wydajność w podglądzie Flutter 22
- 5.7 Pamięć GPU i zwalnianie pamięci GPU 22
- 5.8 Modele jednocześnie 23
- 5.9 Modele jednocześnie w podglądzie Flutter 23
- 5.10 Używany przez inne aplikacje 23

6 **Modele** 25
- 6.1 Pobieranie z Hugging Face 25
- 6.2 Przerwane pobierania w podglądzie Flutter 26
- 6.3 Zapytaj agenta, który model pasuje 26
- 6.4 Zainstalowane 26
- 6.5 Rozmiar kontekstu 26
- 6.6 Na tym komputerze 27
- 6.7 Przenoszenie i usuwanie modeli 27
- 6.8 Jeden folder modeli 27

7 **Łączenie przez rysowanie** 28
- 7.1 Agent ↔ lokalny LLM 28
- 7.2 Agent ↔ serwer MCP 28
- 7.3 Agent ↔ folder skilli 28
- 7.4 Znalezione na tym komputerze 28
- 7.5 Lokalne połączenia w podglądzie Flutter 29
- 7.6 Znalezione na tym komputerze w podglądzie Flutter 29
- 7.7 Gdy połączenie się nie uda 30

8 **Serwery MCP** 31
- 8.1 Skąd pochodzą serwery 31
- 8.2 Konfiguracja serwera 31
- 8.3 Serwery MCP w podglądzie Flutter 32
- 8.4 Zarządzanie serwerami 32

9 **Skille** 33
- 9.1 Znajdowanie skilli 33
- 9.2 Skille z sieci w podglądzie Flutter 33
- 9.3 Foldery skilli 33

10 **Ustawienia i pomoc** 34
- 10.1 Motywy i światła mapy Flutter 34
- 10.2 Tryb okna i okno wyskakujące w Omarchy 34
- 10.3 Dane o użyciu i narożniki z Omarchy w podglądzie Flutter 34
- 10.4 Ustawienia 34
- 10.5 Czerwone limity w podglądzie Flutter 36
- 10.6 Ostatnie zmiany i Cofnij po ponownym uruchomieniu 36
- 10.7 Pomoc 37
- 10.8 Pomoc w podglądzie Flutter 37
- 10.9 Miejsce na dysku 37

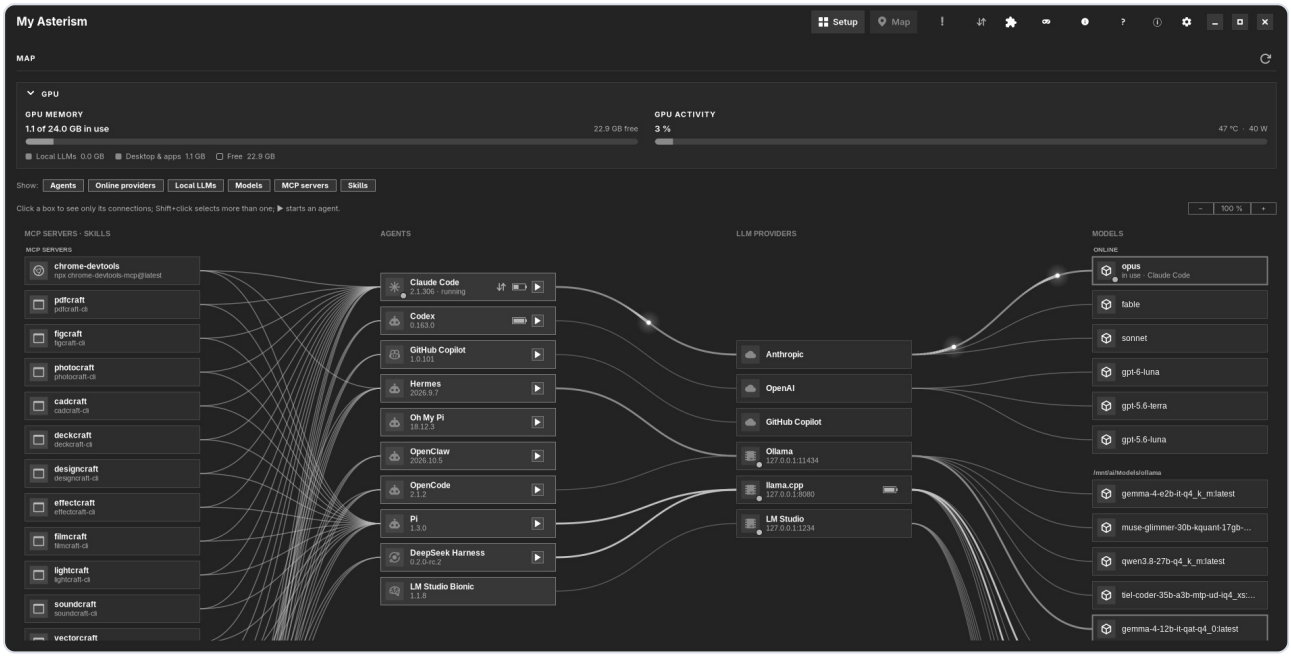
11 **Smart Check** 39
- 11.1 Podsumowanie konfiguracji PC 40
- 11.2 Znalezione aktualizacje 40
- 11.3 Gdy agent pracuje 40
- 11.4 Lokalny pomocnik 41

11.5 Kontrole AI i lokalny pomocnik w podglądzie Flutter 41
11.6 Protokół 41
11.7 Statystyki tokenów 42
12 Karty grup i dodatki na mapie 43
12.1 Rozwijanie kategorii 43
12.2 Zarządzanie dodatkami 43
12.3 Pierwsze użycie i istniejące funkcje 43
13 Edycja mapy SI we Flutterze 44
14 Karty zadań na mapie 45
15 Kontrola stanu i bezpieczeństwa 46
15.1 Czytanie kart 46
15.2 Zmiana ustawienia 47
15.3 Bezpieczeństwo w podglądzie Flutter 47
15.4 Stan w podglądzie Flutter 47
15.5 Co jest sprawdzane u każdego agenta 48
15.6 Panowanie nad ryzykiem 48
15.7 Słowa objaśnione 49
16 Dodatki 51
16.1 Konfiguracja Maprisma 51
16.2 Strona dodatków 52
16.3 Instalowanie, włączanie i wyłączanie, aktualizowanie, usuwanie 53
16.4 Poziomy prywatności 53
16.5 Rodzaje dodatków 54
16.6 Nowy agent na twoim komputerze 54
16.7 Tworzenie dodatku 54
17 Prywatność i bezpieczeństwo 55
18 Rozwiązywanie problemów 56
A Informacje dodatkowe 57
A.1 Skróty klawiszowe 57
A.2 Obsługa klawiaturą w podglądzie Flutter 57
A.3 Pliki i foldery 57
A.4 Słowniczek 58

ROZDZIAŁ 1

Wprowadzenie

Maprisma pokazuje konfigurację AI twojego komputera jako karty i jako mapę: którzy agenci programistyczni są zainstalowani, jakich modeli językowych używają — usług w chmurze i lokalnych środowisk —, jakie modele one udostępniają oraz jakie serwery MCP i skille ma każdy agent. I pozwala zmieniać tę konfigurację, rysując linie.



Rysunek 1 · Główne okno: mapa na całą szerokość, nad nią karta GPU, w nagłówku ! otwartych problemów.

1.1 Co robi

- **Agenci programistyczni** — Claude Code, Codex, GitHub Copilot, Cursor CLI, Gemini, OpenCode, Pi, Hermes i inni: ich wersja, plan, limity i użycie, a także czego każdemu brakuje, z poleceniem, które to naprawia.
- **Lokalne LLM** — llama.cpp, Ollama, LM Studio, vLLM, Jan i KoboldCpp: instalacja, pobieranie modeli, ustawianie rozmiaru kontekstu pasującego do karty graficznej i uruchamianie ich razem z agentami, którzy z nich korzystają.
- **Serwery MCP i skille** — znalezione na komputerze lub w sieci, z polecanymi serwerami i wyszukiwaniem w oficjalnym rejestrze MCP.
- **Stan kolumn** — symbol na pasku każdej kolumny, każdy problem z drogą tam, gdzie można go naprawić; do czego nadaje się twoja konfiguracja, jest na stronie powitalnej i w Pomocy.

1.2 Podstawowe pojęcia

Agent	Program, z którym rozmawiasz i który pisze kod lub wykonuje zadania, na przykład Claude Code.
Dostawca LLM	Miejsce, w którym działa model agenta: usługa w chmurze (Anthropic, OpenAI, …) lub lokalne środowisko LLM na tym komputerze (llama.cpp, Ollama, …).
Model	Sam model językowy. Lokalne modele to pliki, zwykle w formacie GGUF.
Serwer MCP	Mały program lub usługa sieciowa, która daje agentom dodatkowe narzędzia — czytanie plików, wyszukiwanie w sieci, sterowanie przeglądarką. MCP to Model Context Protocol.
Skill	Folder z instrukcjami (plikiem `SKILL.md`) i czasem skryptami, który uczy agenta określonego zadania.

1.3 Nic się nie zmienia bez podglądu

Każda zmiana ustawień agenta jest najpierw pokazywana wiersz po wierszu, zanim cokolwiek zostanie zapisane. Stary plik zostaje jako kopia zapasowa, a pasek, który pojawia się po zmianie, ma **Cofnij**. Sama aplikacja nigdy nie działa z uprawnieniami administratora; tylko instalacja agenta dostarczanego jako pakiet systemowy prosi o hasło, w terminalu, który widzisz.

ROZDZIAŁ 2

Instalacja

2.1 Instalacja Maprisma

Maprisma to jeden program dla Linuksa na komputerach x86-64. Nie potrzebuje Pythona i ma własne ikony oraz podpisane dodatki. Wybierz jeden ze sposobów:

- **Arch Linux i Omarchy (AUR):** zainstaluj pakiet `maprisma-bin` swoim pomocnikiem AUR, na przykład `yay -S maprisma-bin`. Umieszcza aplikację w `/usr/lib/maprisma`, polecenie `maprisma` w ścieżce i Maprisma w menu aplikacji.
- **AppImage, dla innych dystrybucji:** pobierz `Maprisma-<wersja>-x86_64.AppImage` z `github.com/maprisma/maprisma/releases`, nadaj mu prawo wykonywania (`chmod +x Maprisma-*.AppImage`) i uruchom. Korzysta z GTK 3, które ma niemal każdy pulpit Linuksa.

> **UWAGA** AppImage działa z folderu tymczasowego, więc nie konfiguruje usługi w tle dla Ustawienia › **Uruchamiaj lokalne LLM razem z agentami**. Wybierz pakiet AUR, jeśli lokalne LLM mają startować i zatrzymywać się razem z agentami.

Programy opcjonalne: `xdg-terminal-exec` uruchamia agentów we własnym terminalu, `xdg-utils` otwiera podręcznik i strony WWW, a `mise` instaluje i aktualizuje agentów.

Maprisma trzyma ustawienia w `~/.config/maprisma`; bezwzględne `XDG_CONFIG_HOME` przenosi ten folder do `$XDG_CONFIG_HOME/maprisma`. Na Omarchy sama korzysta z integracji z pulpitem: `maprisma --toggle` otwiera okno jako okienko pod paskiem, wysuwa je na wierzch albo je chowa.

2.2 Pierwsze uruchomienie

Pakiet i AppImage nie wymagają dalszej konfiguracji. Jeśli używałeś Maprisma pod dawną nazwą Asterism, pierwsze uruchomienie jednorazowo kopiuje foldery Asterism; oryginały pozostają nietknięte.

Okno jest zwykłym oknem aplikacji: może być kafelkowane, pływające, przeniesione na inny obszar roboczy i dostępne przez `Alt`+`Tab`. MAPRISMA Nowy Maprisma otwiera się na swojej **stronie powitalnej**: agenci i lokalne LLM zainstalowane na tym komputerze to karty już zaznaczone, a **Włącz** umieszcza je na mapie (zob. Konfiguracja Maprisma). OMARCHY Przycisk na pasku otwiera okno, przenosi je na wierzch lub ukrywa, jeśli już jest na wierzchu. Zamknięcie ukrywa okno; program działa dalej w tle, aby otworzyć się natychmiast.

> **UWAGA** Gdy w oknie działa agent, zamknięcie najpierw pyta: **Zatrzymaj i zamknij** (co zatrzymuje też lokalny LLM agenta i zwalnia jego pamięć graficzną) lub **Niech działa**.

Gdy przy pierwszym otwarciu aplikacja sprawdza, co jest na tym komputerze, animowana nakładka pokazuje, co znajduje, rodzaj po rodzaju: agenty, lokalne LLM, serwery MCP i skille, każdy ze wskaźnikiem, który zmienia się w ✓ i liczbę, oraz ile z nich nie jest jeszcze połączonych. Po chwili znika

sama; kliknięcie zamyka ją od razu. Okno otwiera się od razu, z tym przeglądem jako ramką na środku; mapa wypełnia się za nią.

2.3 Usuwanie aplikacji

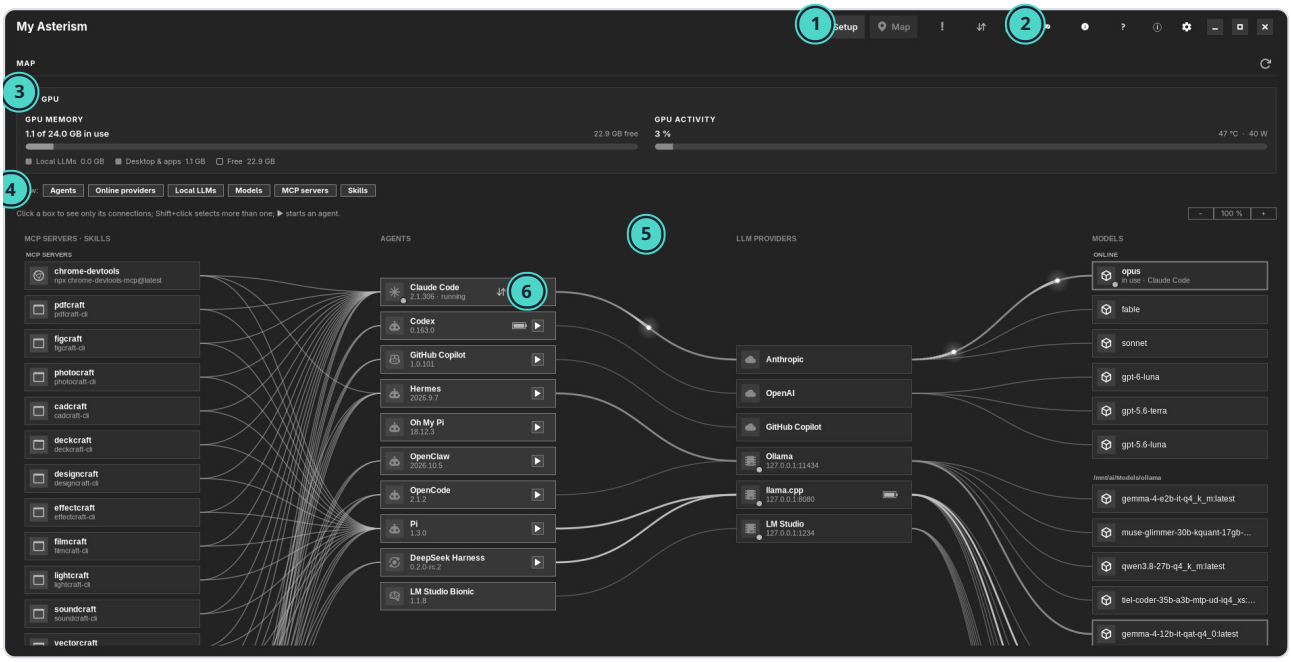
AUR: `sudo pacman -R maprisma-bin`. AppImage: usuń plik.

Przed usunięciem wyłącz Ustawienia › **Uruchamiaj lokalne LLM razem z agentami**, aby usunąć też strażnika działającego w tle. To, co zmieniłeś aplikacją, zostaje: ustawienia agentów, serwery MCP, skille i zainstalowane lokalne LLM. Najpierw odinstaluj lokalny LLM (prawy klik › **Odinstaluj…**), jeśli ma zniknąć. Aby usunąć także własny stan aplikacji:

```
rm -rf ~/.config/maprisma ~/.cache/maprisma \
       ~/.local/share/maprisma ~/.local/state/maprisma
```

ROZDZIAŁ 3

Okno w skrócie



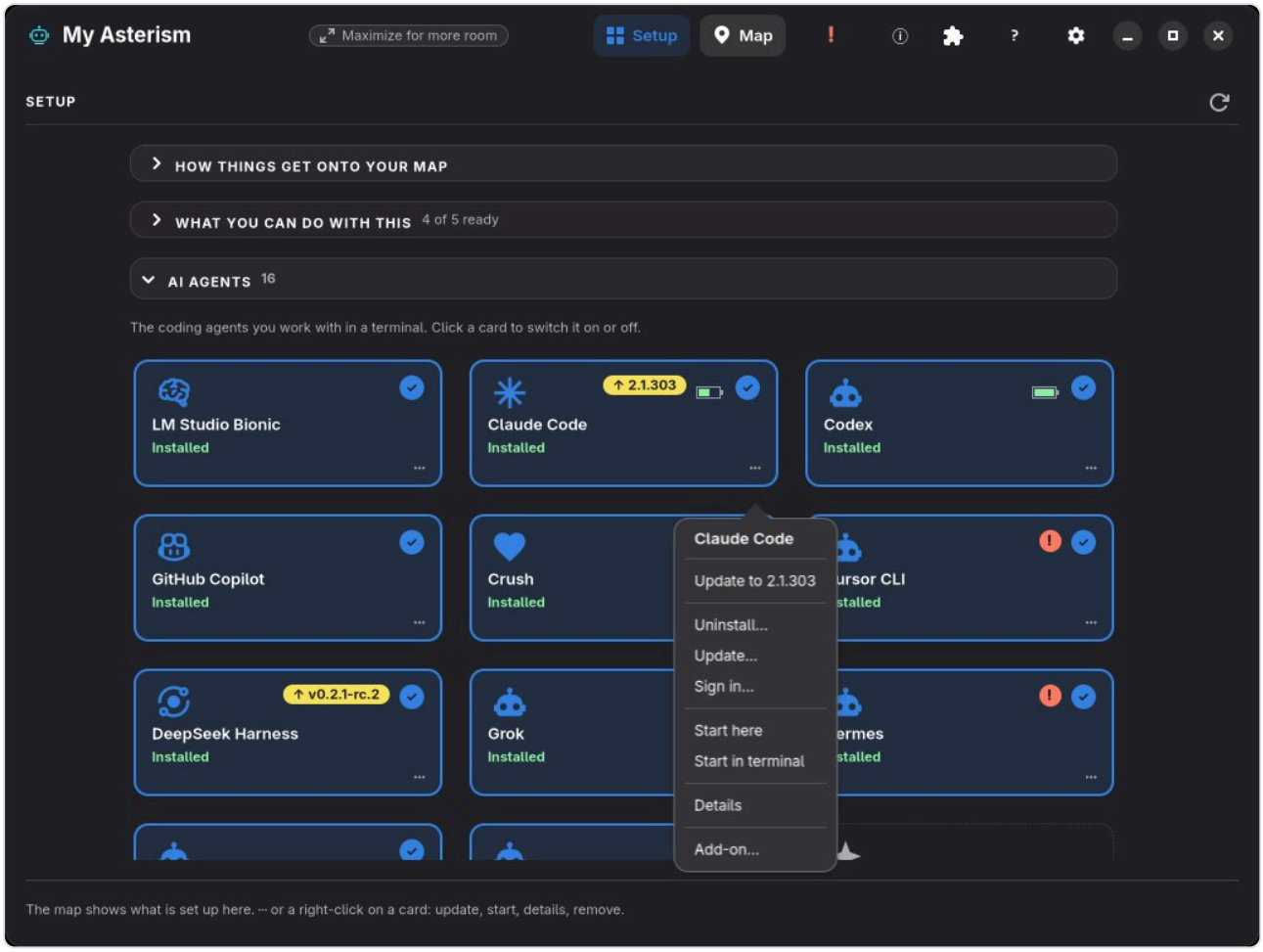
Rysunek 2 · Główne okno. Numery odpowiadają liście poniżej.

1. **Konfiguracja** i **Mapa** — dwa widoki, na każdej stronie; widoczny jest podświetlony. Konfiguracja ma każdego agenta, lokalny LLM, serwer MCP, folder skilli i model jako kartę: instalacja, aktualizacja, logowanie, usuwanie (patrz niżej).
2. Symbole nagłówka: ↻ odśwież (`F5`: sprawdź ponownie, jak przy starcie), **!** stan ogólny (zielony: wszystko w porządku; żółty: wskazówki lub aktualizacje; czerwony: coś do naprawienia; kliknięcie prowadzi do pierwszej kolumny, której to dotyczy), ⓘ legenda (co znaczy każda kropka, linia i znak na mapie), ⇅ ruch sieciowy działających agentów (zob. Ruch sieciowy), dodatki MAPRISMA, wypełnione **i** dla Informacji (zob. Pomoc), **?** Pomoc (`F1`) i zębatka Ustawień (`Ctrl`+`,`). Przytrzymaj lewy przycisk myszy na symbolu i przeciągnij go w bok, aby postawić go gdzie indziej: pasek pokazuje, gdzie trafi, a kolejność zostaje zapamiętana.
3. **GPU** — pamięć GPU i aktywność GPU: jak karta graficzna jest teraz używana, w karcie, która się zwija (zwinięta, jej wiersz zachowuje liczby).
4. Przełączniki **Pokaż** i filtry kolumn (**MCP i skille**, **Agenci**, **Dostawcy**, **Modele**) — ograniczają mapę do części, nad którymi pracujesz.
5. **Mapa** — serwery MCP i skille ← agenci → dostawcy LLM → modele, na całą szerokość, pod nagłówkiem **MAPA**.
6. ▶ w polu agenta uruchamia go w terminalu w oknie; gdy tam działa, przycisk przywraca jego terminal.

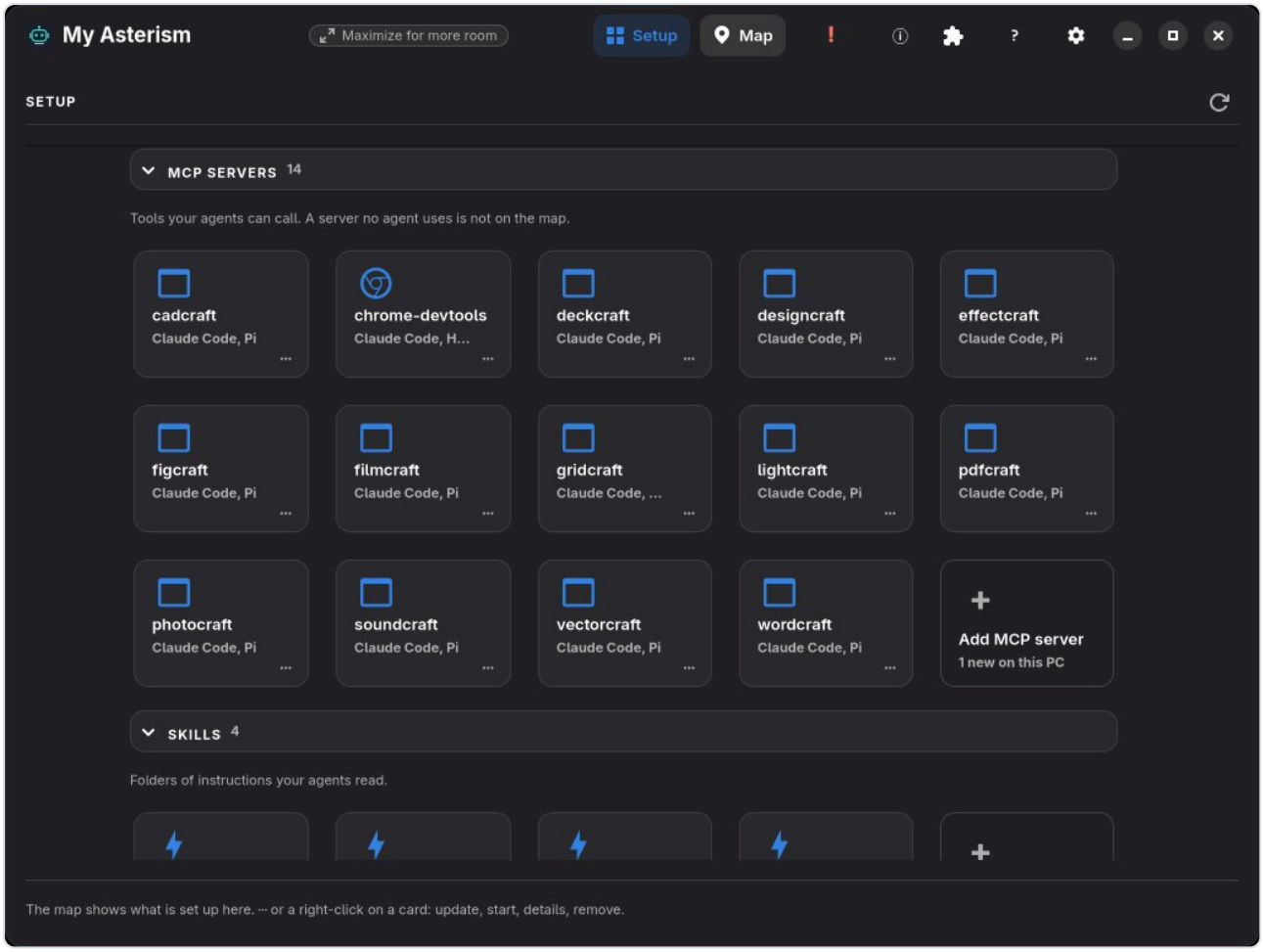
3.1 Konfiguracja

Mapa pokazuje to, co jest skonfigurowane — zainstalowanych agentów i lokalne LLM, serwery MCP i foldery skilli, których używają agenty, oraz modele — a także, wyblakłe, serwery MCP i foldery skilli znalezione na tym komputerze, których żaden agent jeszcze nie używa, gotowe do połączenia linią (zob. Znalezione na tym komputerze). Wszystko inne robi się w sekcji **Konfiguracja** (przycisk w nagłówku, obok **Mapa**), jedna karta na element, w sekcjach, które zwijają się kliknięciem w ich pasek:

- **Jak elementy trafiają na mapę** — zwinięta karta na samej górze; otwórz ją, aby zobaczyć pięć ponumerowanych kroków: włączenie (lub znalezienie) tego, czego używasz, instalacja tego, czego brakuje, logowanie do agenta online, łączenie przez rysowanie i to, co pojawia się samo.
- **Co możesz z tym zrobić** — do czego nadaje się konfiguracja (codzienna pomoc, programowanie, obrazy, głos, research i pisanie), po jednym kafelku z tym, co jest, i następnym krokiem; zwinięta, jej wiersz mówi, ile jest gotowych.
- **Agenci AI, Lokalne LLM** — wszyscy, których zna aplikacja. Karta z niebieską ramką jest zainstalowana na tym komputerze; szara nie, i ma **Zainstaluj** tam, gdzie zainstalowane mają **Odinstaluj**. OMARCHY Kliknięcie zainstalowanej karty otwiera jej szczegóły. MAPRISMA Kliknięcie zaznacza lub odznacza kartę (✓), a potem **Zastosuj** włącza lub wyłącza jej dodatek. Wyłączenie zdejmuje ją z mapy; nic nie jest usuwane.
- **Dostawcy online** MAPRISMA — dodatek dostawców online; jego menu ma **Zaloguj do …** dla każdego zainstalowanego agenta, który z nich korzysta.
- **Serwery MCP, Skille, Modele** — karta dla każdego zainstalowanego elementu. Kliknięcie otwiera szczegóły (w przypadku modelu jego menu). Karta **+** na końcu sekcji dodaje nowy: **Dodaj serwer MCP**, **Dodaj skille**, **Dodaj model**, **Pobierz dodatki**.
- ⋯ lub prawy klik na karcie otwiera to samo menu co pole na mapie: aktualizacja, logowanie, uruchamianie lub zatrzymywanie, szczegóły, odinstalowanie lub usunięcie, przeniesienie modelu.
- Żółte ↑ **wersja** na karcie oznacza, że czeka aktualizacja; jej menu ma **Aktualizuj**.
- Czerwony **!** na karcie oznacza tam otwarty problem; najedź na niego, aby zobaczyć jaki.
- Szczegóły agenta pokazują pod planem **Dodatkowe użycie**: czy poza limitami planu może kosztować dodatkowe pieniądze. Claude Code podaje to na Twoim komputerze (wraz ze stroną do zmiany); przy innych agentach aplikacja mówi, że tego nie wie, więc sprawdź to u dostawcy.
- Mała bateria na karcie agenta online pokazuje, ile zostało z jego limitów (sesja, tydzień): liczy się najciaśniejszy, żółta przy jednej czwartej lub mniej, czerwona poniżej jednej dziesiątej; najedź na nią, aby zobaczyć każdy limit i kiedy się odnawia. Na karcie działającego lokalnego LLM bateria to wciąż wolna pamięć GPU, w tych samych kolorach; najedź na nią, aby zobaczyć, ile zajmuje ten lokalny LLM.
- Symbol odświeżania (F5) w pasku nagłówka sprawdza ponownie, co jest zainstalowane, skonfigurowane i zalogowane, i powtarza sprawdzenia ze startu (nowe aplikacje, dodatki, aktualizacje, otwarte problemy).



Rysunek 3 · Konfiguracja: zainstalowani agenci mają niebieską ramkę, włączeni ✓, aktualizacje żółtą, a problemy czerwoną plakietkę; menu Claude Code.



Rysunek 4 · Serwery MCP i skille, każda sekcja z kartą do dodawania.

Nie ma nic do ukrywania: to, czego nie chcesz na mapie, wyłączasz lub usuwasz tutaj. Gdy instalacja lub logowanie działa w terminalu w oknie, **Wstecz** (lub **×** na jego karcie) wraca na stronę, z której je uruchomiono.

Lokalne serwery MCP i **Serwery MCP online** (osiągalne pod adresem internetowym) to dwie sekcje. **Dodaj folder...** w sekcjach **Lokalne serwery MCP** i **Skille** dodaje folder, w którym aplikacja również ich szuka — na przykład wspólny dysk z rozpakowanymi aplikacjami, które oferują serwer MCP, albo folder z własnymi skillami. Dodane foldery są wymienione obok przycisku; **×** przy jednym kończy szukanie w nim (nic nie zostaje usunięte). To, co w nich zostanie znalezione, pojawia się wyblakłe na mapie, dopóki linia tego nie połączy: Konfiguracja ma karty tylko dla tego, co jest skonfigurowane.

3.2 Wybór, co pokazuje mapa

Mapa pokazuje wszystko, co jest skonfigurowane. Przełączniki **Pokaż** pod kartą GPU oraz przyciski filtrów kolumn (**MCP i skille**, **Agenci**, **Dostawcy**, **Modele**) nad mapą w aplikacji Flutter pomijają całe obszary lub kolumny i są zapamiętywane.

W aplikacji Flutter widok mapy zależy od części ekranu zajmowanej przez okno (lub od szerokości okna, gdy rozmiar ekranu nie jest znany): **Mały** (do 55 % ekranu lub poniżej 1150 pikseli) pokazuje **Agenci** i **Dostawcy**; **Średni** (do 80 % lub poniżej 1600 pikseli) dodaje **Modele**; a **Maksymalny** (powyżej 80 % lub od 1600 pikseli) pokazuje wszystkie cztery kolumny, w tym **MCP i skille**. Kolumnę pominiętą przez

widok można mimo to włączyć jej przyciskiem nad mapą, a mapa ułoży się na nowo. W każdym widoku pokazywane kolumny są wyśrodkowane i dopasowują swoją szerokość (266 do 560 pikseli) oraz odstęp między kolumnami do wolnego miejsca.

3.3 Praca z mapą

- **Najedź** na pole, aby podświetlić je razem ze wszystkim, z czym jest połączone: ich ramki przyjmują kolor akcentu, a reszta mapy przygasa. W przeciwnym razie każde pole ma taką samą cienką szarą ramkę.
- **Kliknij** pole, aby je zaznaczyć (✓); `Shift`+klik dodaje kolejne. Zaznaczone pola i wszystko, z czym są połączone, zachowują ramkę w kolorze akcentu, dopóki wskaźnik nie jest nad innym polem. `Esc` lub **Wyczyść zaznaczenie** je czyści.
- **Pinezka** w prawym górnym rogu pola utrzymuje podświetlenie jego połączeń, gdy patrzysz gdzie indziej.
- **Prawy klik** na polu daje wszystko, co oferuje jego karta w Konfiguracji: uruchamianie lub zatrzymywanie, szczegóły, instalację, logowanie, modele, aktualizację, usuwanie i więcej.
- ▶ w polu agenta uruchamia go w terminalu w oknie.
- Agenci zawsze pozostają na środku widocznej części mapy podczas przewijania.
- Aktywna ścieżka każdego agenta — agent → używany dostawca → jego model — jest rysowana pogrubioną linią. Gdy agent pracuje, wzdłuż tej linii biegnie światło.
- **Pole statystyk użycia**: gdy agent pracuje, wzdłuż jego linii do dostawcy pojawiają się małe liczby w kolorze linii: jak szybko przyszła jego ostatnia odpowiedź (**t/s**, tokeny na sekundę), rozmiar jego kontekstu (**1M**) i ile z niego jest w użyciu (**%**, żółty od 75 %, czerwony powyżej 90 %, jak przy bateriach). Pochodzą z własnych plików sesji agenta (tylko liczby; Claude Code, Codex i Pi). Co pokazuje i rozmiar tekstu ustawia się w Ustawieniach.
- Czerwona kropkowana linia łączy agenta z dostawcą, dla którego jest skonfigurowany, ale którego teraz nie używa (żaden jego model nie jest w użyciu).
- Gdy agenci używają różnych modeli jednego lokalnego LLM, linie do tych modeli są fioletowe: trzyma je załadowane jednocześnie albo je przełącza (zob. Modele jednocześnie).
- Pole agenta online ma tę samą baterię co jego karta w Konfiguracji: ile zostało z jego limitów; pole działającego lokalnego LLM — wciąż wolną pamięć GPU.
- Agent płatny za token — zalogowany kluczem API, więc bez limitów planu na baterię — pokazuje zamiast tego, ile kosztują dziś w cenach API jego aktywne sesje, jako małe **„$3.40 dziś”** na polu i karcie: zielone, żółte od 75 % dziennego limitu i czerwone powyżej (Ustawienia › **Koszty**), odświeżane co minutę. Jego szczegóły pokazują cały koszt API.
- Gdy program spoza mapy używa lokalnego LLM, linia od tego LLM do jego modelu staje się gruba i ma własny kolor (fioletowy lub cyjanowy, ten, który bardziej odróżnia się od pozostałych kolorów motywu), a wzdłuż niej biegnie światło. Zob. Używany przez inne aplikacje.
- Gdy agent uruchomił innego — na przykład Claude Code wykonujący przez swoją powłokę `pi -p …` lub `codex exec …` — wzdłuż lewej strony karty agentów biegnie **strzałka** od agenta, który używa drugiego, do używanego, a po niej biegnie światło; jej podpowiedź wymienia obu. Agent może używać kilku innych naraz, każdego z własną strzałką. Dopóki jest używany, używany agent ma też swoje linie do dostawcy i dalej do modelu w kolorze strzałki, a światło biegnie po nich w tym kolorze: od razu widać, który dostawca i model pracuje dla kogo.

- Symbol odświeżania (F5) w pasku nagłówka sprawdza ponownie, co jest zainstalowane, skonfigurowane i zalogowane, i powtarza sprawdzenia ze startu (nowe aplikacje, dodatki, aktualizacje, otwarte problemy).
- Dopóki nie zostanie narysowana pierwsza linia, karta nad mapą pokazuje w trzech krokach, jak to zrobić; **Zobacz jak** odtwarza animację z Pomocy. Po pierwszym połączeniu, albo po kliknięciu ×, znika na zawsze.
- Mapa dopasowuje się do szerokości okna (30 do 100 %), dopóki nie zmienisz powiększenia przez **− 100 % +** w prawym górnym rogu lub Ctrl++ / -; Ctrl+0 ponownie dopasowuje ją do okna. Kółko myszy przewija mapę.
- Pola leżą w jednej karcie na kategorię (agenci, dostawcy online, lokalne LLM, karta na każdy folder modeli, serwery MCP, skills). Przeciągnij kartę za nagłówek, aby zmienić kolejność w jej kolumnie, a pole w jego karcie, aby zmienić kolejność.
- **Kliknij nagłówek karty**, aby zwinąć ją do nagłówka (▸ po prawej stronie nagłówka), i jeszcze raz, aby ją rozwinąć (▾). Linie pól zwiniętej karty kończą się przy jej nagłówku, więc nadal widzisz, z czym jest połączona (strzałki między agentami są ukryte, dopóki karta agentów jest zwinięta). Rozwinięcie karty przesuwa w dół karty znajdujące się pod nią w jej kolumnie.
- **Pinezka** na prawo od powiększenia zachowuje układ mapy — kolejność kart i pól oraz to, które karty są zwinięte — po **Odświeżeniu** (F5) i ponownym uruchomieniu. Bez przypięcia, jak na początku, mapa wraca do domyślnego układu przy każdym odświeżeniu i starcie; twoje zmiany trwają do tego czasu. **Układ domyślny** przywraca wszystko naraz.
- **Wyblakłe pola** na kartach serwerów MCP i skilli są na tym komputerze, ale żaden agent ich nie używa. Narysuj linię od agenta do jednego z nich, aby je połączyć (zob. Znalezione na tym komputerze). Serwery MCP osiągalne pod adresem internetowym mają własną kartę, **Serwery MCP online**, obok karty **Lokalne serwery MCP**; obie zwijają się jak każda inna karta.

> **WSKAZÓWKA** W wąskim oknie (do 55 % ekranu lub poniżej 1150 pikseli) mapa pokazuje kolumny Agenci i Dostawcy; poszerz okno lub włącz Modele oraz MCP i skille przyciskami nad mapą.

3.4 Stan kolumn

Pasek **Zapytaj AI…** każdej kolumny pokazuje po lewej jej stan, a nagłówek pokazuje najgorszy stan wszystkich kolumn, także ukrytej: zielony znacznik, gdy wszystko jest w porządku, żółty trójkąt przy wskazówkach, aktualizacjach lub sugestiach, czerwony **!**, gdy coś trzeba naprawić. Kliknięcie symbolu otwiera listę kolumny, najpoważniejsze najpierw: niezalogowany agent, lokalny LLM bez modeli, niedziałające połączenie, mało miejsca na modele, każde z przyciskiem prowadzącym tam, gdzie można to naprawić. **Zapytaj AI** przy wierszu wstawia go do pytania; dopiero **Pytaj** je wysyła. Kliknięcie symbolu w nagłówku prowadzi do pierwszej kolumny, której to dotyczy. Sprawdzanie tylko odczytuje; niczego nie zmienia, nie uruchamia ani nie wysyła.

Lista pokazuje też nowe programy lub skille znalezione na tym komputerze, to, co jest na tym komputerze, ale nie na mapie, oraz aktualizacje zainstalowanych agentów, lokalnych LLM i serwerów MCP, każde ze swoim przyciskiem. Aktualizacje są sprawdzane w tle przy otwarciu okna, najwyżej co sześć godzin. Aktualizacja zaczyna się dopiero po kliknięciu **Aktualizuj** (zobacz Znalezione aktualizacje). Do czego nadaje się twoja konfiguracja, jest na stronie powitalnej i w Pomocy, w **Co możesz z tym zrobić**.

3.5 Inteligentne karty na mapie

W podglądzie Flutter na górze każdej kolumny mapy znajduje się zwinięty pasek **Zapytaj AI…** z symbolem stanu po lewej stronie: zielonym znacznikiem, gdy wszystko jest w porządku, żółtym trójkątem dla wskazówek, aktualizacji lub propozycji, albo czerwonym **!**, gdy w tej kolumnie jest coś do naprawy. Pasek nagłówka pokazuje ogólny stan wszystkich czterech kolumn (także ukrytych) i otwiera pierwszą kolumnę wymagającą uwagi. Kliknięcie symbolu stanu kolumny otwiera jej listę stanu: każdy punkt do naprawy dla tej kolumny z przyciskiem otwierającym miejsce naprawy oraz **Zapytaj AI** do wstępnego wypełnienia zapytania, nowsze wersje zainstalowanych agentów i lokalnych LLM (**Aktualizuj** uruchamia `mise upgrade` albo własny aktualizator agenta w terminalu lub otwiera podgląd instalacji; sprawdzane najwyżej co sześć godzin), aktualizacje serwerów MCP z codziennych kontroli AI oraz **Sprawdź ponownie**. Na stronie powitalnej **Co możesz z tym zrobić** pokazuje zastosowania (codzienna pomoc, vibe coding, obrazy, głos, research i pisanie) jako **Gotowe**, **Prawie** lub **Jeszcze nie**, z tym, co jest do każdego, i następnym krokiem.

Przy starcie aplikacja czyta w tle podpisany katalog dodatków: z pamięci podręcznej, a online tylko wtedy, gdy włączone jest **Szukaj dodatków online**. Lista stanu każdej kolumny pokazuje wtedy także agentów i lokalne LLM na tym PC, których dodatek jest wyłączony lub jeszcze niezainstalowany (**Pobierz dodatki** lub **Dodatki** otwiera dodatki do włączenia lub pobrania), nowe programy lub skille znalezione na tym PC (**Dodaj serwer MCP…**, **Dodaj skille…**) oraz zainstalowane dodatki z nowszą wersją w katalogu. **Zaktualizuj wszystko** pokazuje je z wersjami i po potwierdzeniu aktualizuje jeden po drugim, każdy sprawdzony z katalogiem; jeden komunikat mówi, co zaktualizowano, a czego nie.

Kliknięcie części **Zapytaj AI…** paska otwiera pole zapytania nad tą kolumną bez przesuwania mapy (albo oferuje **Zacznij**, gdy żaden pomocnik AI nie jest jeszcze skonfigurowany). Zanim cokolwiek zostanie wysłane, karta pokazuje, który pomocnik odpowiada (z menu, jeśli gotowych jest więcej) oraz czy działa lokalnie (**Lokalnie, bez tokenów**), czy **Używa planu/tokenów**; dopiero kliknięcie **Zapytaj** wysyła zapytanie. Pomocnik zadaje jedno pytanie doprecyzowujące, proponuje przeniesienie zapytania do właściwej kolumny albo wskazuje sprawdzonych kandydatów z rzeczywistych wyników wyszukiwania. W kolumnie **Modele** karta pokazuje VRAM, RAM i wolne miejsce na tym PC oraz oznacza każdego kandydata jako **Mieści się w VRAM**, **Używa odciążenia RAM** lub **Za duży dla tego PC**. **Sprawdź** otwiera podgląd pobierania przed pobraniem lub zmianą czegokolwiek, a gdy model jest gotowy, możesz opcjonalnie połączyć go z agentem przez podgląd połączenia z opcją **Cofnij**.

W kolumnie **Serwery MCP i skille** pomocnik może dodać serwer MCP z oficjalnego rejestru, połączyć serwer z agentem lub go odłączyć. Wybierasz agentów, wypełniasz to, czego serwer potrzebuje (klucze zostają na tym komputerze i nigdy nie trafiają do pomocnika), i zatwierdzasz zwykły przegląd; wtedy zmiana jest wykonywana, a mapa ją pokazuje. Wszystko inne odrzuca i wskazuje właściwą kolumnę. Nie potrafi jeszcze zarządzać skillami, aktualizować programów ani usuwać serwera z komputera.

W kolumnie **Modele** pomocnik może też usunąć model lokalnego LLM: najpierw widzisz zwykły przegląd, a pliki trafiają do Kosza, gdy to możliwe. Może też przenieść folder modeli na inny dysk, z podglądem i **Cofnij** w nagłówku. Każdy folder ma własny przegląd.

W kolumnie **Agenci** pomocnik może zainstalować, zaktualizować lub usunąć agenta, jeśli strona Agenci to oferuje. Widzisz zwykły przegląd; krok działa potem w terminalu, który możesz obserwować. Nie uruchamia agentów ani nie zmienia ich ustawień.

W kolumnie **Dostawcy** pomocnik może zainstalować lub zaktualizować lokalny LLM oraz go uruchomić lub zatrzymać, jeśli aplikacja to oferuje; otwiera się zwykły przegląd aplikacji. Nie może odinstalować dostawcy ani zalogować się do dostawcy online.

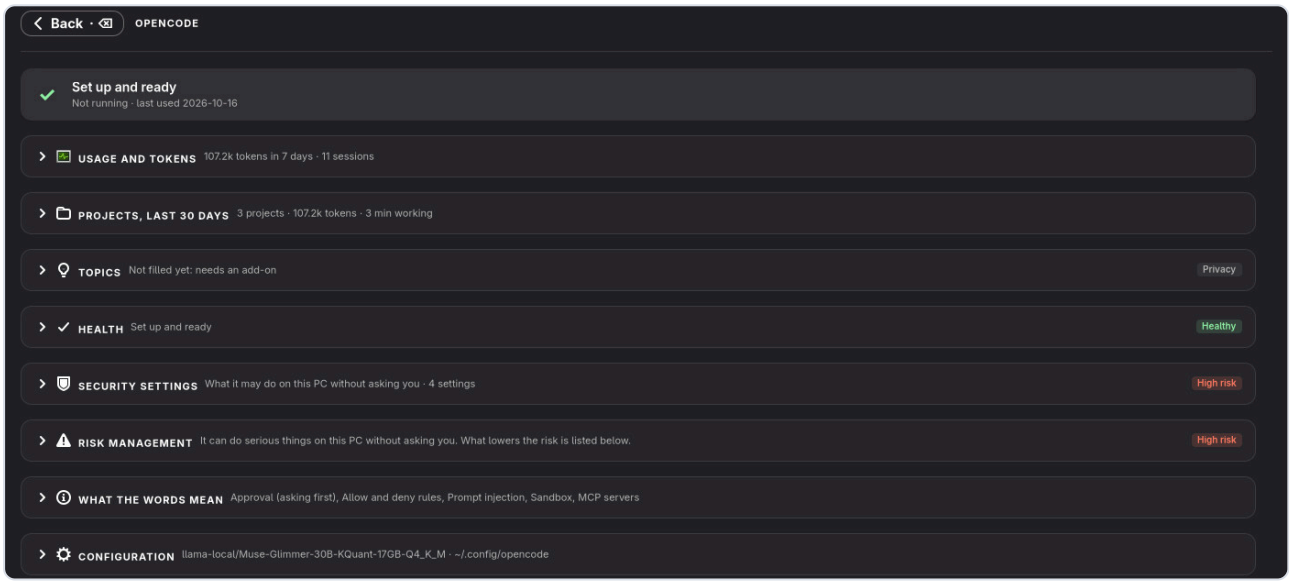
Gdy kolumna czegoś nie potrafi, ale sieć może to wyjaśnić, pomocnik online szuka w sieci, a karta pokazuje jego odpowiedź ze źródłami; lokalny pomocnik nie potrafi szukać w sieci i to mówi. Po takiej odpowiedzi **Zrób to z …** pokazuje polecenie, a agent pracuje w terminalu i pyta przed każdym krokiem zmieniającym pliki.

ROZDZIAŁ 4

Agenci

4.1 Szczegóły agenta

Każdy agent programistyczny ma kartę w Konfiguracji, zainstalowani z wersją. Kliknij kartę zainstalowanego agenta (MAPRISMA lub ··· › **Szczegóły**) albo prawym przyciskiem jego pole na mapie › **Szczegóły**. Na górze wiersz mówi, czy agent jest gotowy (a jeśli nie, dlaczego) i czy teraz działa, lub kiedy był ostatnio używany. Poniżej każda część jest zwijaną kartą: kliknięcie nagłówka ją otwiera lub zamyka. Zaczynają zwinięte, każda z jednowierszowym podsumowaniem i plakietkami; część, którą otworzysz, pozostaje otwarta na kolejnych stronach szczegółów.



Rysunek 5 · Szczegóły agenta: każda część zwinięta, z podsumowaniem i plakietkami.

Karta	Co pokazuje
Użycie i tokeny	To, co agent zapisuje: tokeny i sesje z dziś i z ostatnich siedmiu dni, plan, wykres tokenów na dzień, limity z czasem resetu i tokeny według modelu.
Koszt API / **Wartość API**	Ile kosztują jego tokeny w cenach API dostawców: dziś, w ostatnich 7 i 30 dniach oraz dla każdego modelu podział na wejście, wyjście, odczyt i zapis pamięci podręcznej (tokeny i kwota). **Koszt API** dla agenta płatnego za token (z jego udziałem w dziennym limicie), **Wartość API** dla agenta z planem (plan to pokrywa). Zobacz Koszt API.
Projekty, ostatnie 30 dni	Dla każdego folderu projektu: czas pracy, sesje, tokeny, uruchomienia subagentów, zmienione wiersze i wartość API.
Tematy (Prywatność)	O czym były sesje. To wykracza poza liczby, dlatego karta ma oznaczenie **Prywatność** i wypełnia ją tylko dodatek (zob. niżej); dodatek nazywa kartę, a jej plakietka mówi, ile czyta.

Karta	Co pokazuje
Stan	Każde sprawdzenie konfiguracji z ✓ albo co zrobić dalej: zalogować się, zainstalować lub uruchomić lokalny LLM, pobrać model, dodać serwery MCP lub skille.
Ustawienia bezpieczeństwa, **Zarządzanie ryzykiem**, **Co znaczą te słowa**	Co agent może zrobić bez pytania i co zmniejsza ryzyko (zob. Kontrola stanu i bezpieczeństwa).
Konfiguracja	Ustawienia agenta, serwery MCP, skille i jego folder ustawień.

Projekty są pokazywane dla Claude Code, Codex, Pi, Oh My Pi, OpenCode, Hermes, DeepSeek Harness, Goose, Gemini CLI i OpenClaw: czas pracy, sesje, tokeny, uruchomienia subagentów, zmienione wiersze i ile kosztowałyby tokeny w cenach API. Liczby pochodzą z własnych plików sesji lub bazy danych każdego agenta; żaden tekst nie jest czytany.

Koszt API. Kwoty są szacunkiem: liczby tokenów prowadzone przez agenta (wejście, wyjście, odczyt i zapis pamięci podręcznej, dla każdego modelu) w opublikowanych cenach API dostawców — Anthropic, OpenAI, DeepSeek, xAI i Google, z datą ich pobrania. Sesja liczy się w dniu ostatniej odpowiedzi; modele lokalne nic nie kosztują. Crush nie prowadzi sumowalnych liczników tokenów, więc jego karta pokazuje koszt podany przez sam Crush. Dla agenta, który nie prowadzi na PC liczników tokenów (Cursor CLI, Antigravity, Muse), karta mówi, że koszt jest nieznany; ma go strona konta jego dostawcy. Kwoty są w dolarach amerykańskich albo w euro po dziennym kursie referencyjnym Europejskiego Banku Centralnego (Ustawienia › **Koszty**).

MAPRISMA Z dodatkiem **Insights** (zob. Dodatki) karta **Tematy** pokazuje najpierw, które tematy zużyły najwięcej tokenów w ostatnich 30 dniach (**Najwięcej tokenów**: sesje o tym samym tytule to jeden temat, każdy z udziałem we wszystkich tokenach), potem tytuły, które agent sam nadał swoim sesjom, według projektu (**Według projektu**), i oferuje **Podsumuj z Claude…** (gdy Claude Code jest zainstalowany). Jej tokeny pochodzą z własnych plików sesji agenta bez odczytów z pamięci podręcznej, więc mogą być dużo niższe niż w **Użycie i tokeny**, które może liczyć odczyty z pamięci podręcznej. Zanim cokolwiek zostanie wysłane, widzisz dokładnie, co zostanie wysłane: tytuły i liczby, nigdy rozmowy.

4.2 Uruchamianie i zatrzymywanie agentów

Dla każdego agenta zachowywanych jest 50 ostatnich folderów projektów.

Podmenu **Uruchom** każdego agenta ma trzy sekcje oddzielone liniami: **Uruchom** z bieżącym folderem, **Wybierz folder…** oraz listę rozwijaną z wyszukiwaniem **Ostatnie projekty**. Przewijana lista zawiera wszystkie foldery użyte do uruchomienia tego agenta z mapy, od najnowszych. Wybierz poprzedni projekt, a potem Uruchom. Okno wyboru folderu uruchamia jedną sesję w wybranym folderze. Przypięty folder nie jest wymagany: bez niego Uruchom używa Ustawienia › **Agenci uruchamiają się w** lub folderu domowego. Strona szczegółów ma przewijaną kartę **Ostatnie projekty** ze wszystkimi zapisanymi folderami. Kliknięcie wiersza otwiera Uruchom z tym folderem. Pinezka ustawia następny folder domyślny tego agenta; można przypiąć jeden folder na agenta. Odpięcie przywraca ogólny domyślny folder. Kosz usuwa wpis i pinezkę z historii tego agenta, nigdy folder ani

jego pliki. To samo podmenu, historia i pinezki dotyczą **Uruchom tutaj** i **Uruchom w terminalu**. **Uruchom ponownie** znów otwiera podmenu. Działające sesje zachowują folder; anulowane wybory i nieudane uruchomienia nie dodają historii.

- ▶ w polu agenta lub **Uruchom tutaj** w jego menu uruchamia agenta w terminalu w oknie, zamiast mapy. Wymaga VTE (`vte4`).
- **Uruchom w terminalu** otwiera go we własnym oknie terminala.
- Gdy działa w oknie: **Pokaż terminal** i **Zatrzymaj**. Zatrzymanie kończy agenta i wszystko, co uruchomił, na przykład jego serwery MCP.
- **Terminale…** pokazuje, gdzie jeszcze działa agent: *Tutaj, w tym oknie*, każde *Okno terminala* (kliknięcie przenosi je na wierzch) i każda instancja *W tle* — bez okna, na przykład `claude -p …` uruchomione przez skrypt lub innego agenta. Kliknięcie instancji w tle otwiera kartę terminala, która pokazuje na bieżąco, co ona pisze: pytania, odpowiedzi i używane narzędzia (z jej pliku sesji albo z jej wyjścia, gdy trafia ono do pliku). Karta kończy się wraz z instancją. To jedyne miejsce, w którym aplikacja pokazuje tekst rozmowy, i tylko dla kliknięte instancji.
- **Zatrzymaj…** — także mały przycisk zatrzymania na lewo od ▶ w polu agenta — zatrzymuje jedną z jego instancji poza tym oknem. Wybierz ją z listy, która podaje, gdzie i kiedy każda została uruchomiona. Najpierw jest proszona o zakończenie (SIGTERM), a jeśli po 3 sekundach nadal działa, zostaje zakończona. Zatrzymywany jest tylko ten jeden proces: jeśli w międzyczasie się zakończył, a jego numer procesu należy teraz do innego programu, nic się nie dzieje.

Gdy agenci mają terminale w oknie, rząd kart nad mapą pokazuje jedną kartę na terminal, z kropką oznaczającą *pracuje*, *działa* lub *zakończony*. Kliknięcie karty pokazuje ten terminal; **×** zamyka zakończony.

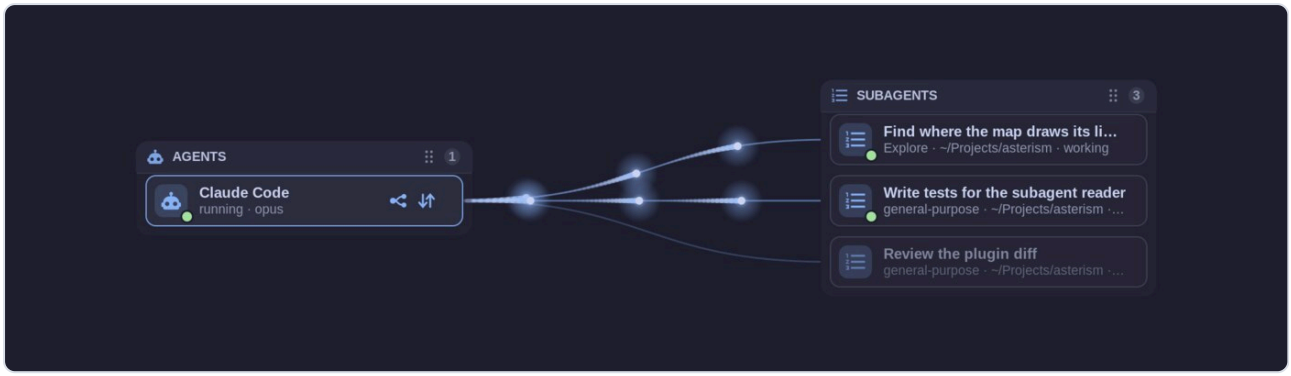
Agent otwarty gdziekolwiek na komputerze dostaje zieloną kropkę i *działa*, a pole modelu, z którym działa, mówi *w użyciu · <agent>*.

4.3 Ruch sieciowy

Pole i karta działającego agenta pokazują, co odbiera i wysyła (↓ do, ↑ z). Kliknij ten wiersz albo ⇅ w nagłówku dla wszystkich działających agentów, aby otworzyć jego stronę sieci: **Teraz**, **Ostatnie 5 minut** jako wykres (przychodzące zielone, wychodzące niebieskie), **Dokąd idzie** — API online, lokalne LLM, ten komputer i twoja sieć lokalna, każde ze swoim udziałem — oraz **Programy**: sam agent i programy przez niego uruchomione, np. jego serwery MCP. Liczone jest z własnych połączeń TCP agenta, gdy aplikacja jest otwarta (bez uprawnień administratora; UDP nie jest liczone), i zostaje na tym komputerze.

4.4 Podagenci

Claude Code i OpenCode mogą przekazywać części zadania **podagentom**. Dopóki któryś z podagentów agenta pracuje, jego pole na mapie pokazuje obok ⇅ mały znak drzewa; gdy na liście są tylko zakończeni, znak jest przygaszony. Kliknięcie go otwiera stronę **Podagenci** agenta: osobną mapę ułożoną jak duża, z kartą agenta po lewej i kartą **Podagenci** po prawej. Każde pole pokazuje zadanie podagenta, jego typ, folder, w którym pracuje jego sesja, oraz **pracuje** albo **zakończony … minut temu**. Po liniach do pracujących biegnie światło; zakończeni są przygaszeni. Najedź na pole, aby przeczytać całe zadanie.



Rysunek 6 · Podagenci agenta: dwóch przy pracy, jeden zakończony.

Jak długo zakończeni podagenci zostają na liście (30 minut) i ilu najwyżej (10), ustawia się w Ustawienia › **Podagenci**; pracujący są zawsze pokazywani. Strona czyta tylko typ każdego podagenta i jednowierszowy opis jego zadania, z własnych plików agenta (folderów sesji Claude Code, bazy danych OpenCode), nigdy tego, co mówią podagenci.

4.5 Współpracujący agenci w podglądzie Flutter

W podglądzie Flutter agent, który uruchomił innego i go używa — na przykład Claude Code wykonujący `pi -p …` przez swoją powłokę — jest z nim połączony na mapie linią wygiętą wzdłuż prawej strony pól agentów. Światło płynie po niej, dopóki któryś z nich pracuje. Rozpoznaje się to tylko z drzewa procesów (własnych procesów agentów i ich rodziców), a linia jest pokazywana, nigdy edytowana. Subagenci Claude Code, Codex, GitHub Copilot, OpenCode i Goose są odczytywani jak w aplikacji.

4.6 Koszty w podglądzie Flutter

Szczegóły agenta pokazują w podglądzie Flutter, ile kosztują jego tokeny po cenach API dostawców: dziś, w ostatnich 7 i 30 dniach oraz dla każdego modelu z podziałem na wejście, wyjście, odczyt cache i zapis cache. **Koszt API** dla agenta płacącego za tokeny (z udziałem w dziennym limicie), **Wartość API** dla agenta z planem (który to pokrywa). To szacunek z liczb w jego własnych plikach sesji; żaden tekst z nich nie jest czytany. Ustawienia › **Koszty** ustala **Walutę** (euro po dziennym kursie referencyjnym Europejskiego Banku Centralnego, pobieranym raz dziennie), **Dzienny limit** oraz progi **Żółty od** i **Czerwony od** (czerwony zawsze powyżej żółtego). Pliki sesji Claude Code są czytane dalej od miejsca, w którym skończyło się ostatnie spojrzenie, więc nawet bardzo długie sesje liczą się w całości; pierwsze spojrzenie na komputerze z bardzo wieloma z nich może wymagać dwóch, by było pełne.

Na mapie agent z limitami planu pokazuje w swoim polu małą baterię z tym, co zostało z najciaśniejszego limitu, a agent płacący za tokeny pokazuje tam zamiast tego dzisiejszy koszt; zielony, żółty lub czerwony według progów z Ustawienia › Koszty, szczegóły w podpowiedzi. Oba są sprawdzane co minutę. Gdy **Powiadom przy czerwonym** jest włączone, koszt na czerwono dostaje jedno powiadomienie na pulpicie dziennie.

4.7 Szczegóły w podglądzie Flutter

Szczegóły agenta pokazują też **Projekty, ostatnie 30 dni**: dla każdego folderu projektu czas pracy, sesje, tokeny, zmienione wiersze i wartość API, uruchomienia subagentów według typu i najbardziej aktywną godzinę, z liczb w jego plikach sesji. Karta lokalnego LLM mówi, czy działa, ile pamięci GPU zajmuje i które modele są załadowane, pokazuje jego modele, agentów i adres, **Użycie i tokeny**, które agenci przez niego wysłali (dopasowane po nazwie modelu), a w **Zarządzanie ryzykiem**, kto może się do niego dostać. Na mapie działający lokalny LLM pokazuje wciąż wolną pamięć GPU jako baterię. Kliknięcie pola serwera MCP lub zestawu skilli otwiera jego szczegóły: którzy agenci go mają i jak każdy go uruchamia, wywołuje lub czyta (wartości kluczy, tokenów i parametrów zapytania nigdy nie są pokazywane), kto go jeszcze nie ma, oraz narzędzia serwera z ostatniego zapytania lub opis każdego skilla.

4.8 Karty terminali, pole użycia i model startowy w podglądzie Flutter

Terminale otwierają się w wyśrodkowanym oknie nad mapą. Dopóki agenci działają w terminalach okna, nad mapą jest karta dla każdego, z kropką oznaczającą **pracuje**, **działa** lub **zakończony**; kliknięcie pokazuje terminal, a widoczny jest zaznaczony (Ustawienia: **Karty terminali nad mapą**). Pole pracującego agenta na mapie pokazuje **Pole statystyk użycia**: jak szybko przyszła jego ostatnia odpowiedź (tokeny na sekundę), rozmiar jego kontekstu i ile z niego jest zajęte (żółty od 75 %, czerwony powyżej 90 %), odczytane z jego własnych plików sesji, tylko liczby; Ustawienia mówią, co jest widoczne i w jakim rozmiarze. W **Twój pierwszy agent** trasa z lokalnym LLM proponuje model startowy pasujący do twojej karty graficznej (**Model**), a **Pobierz go** uruchamia jego pobieranie jak strona modeli.

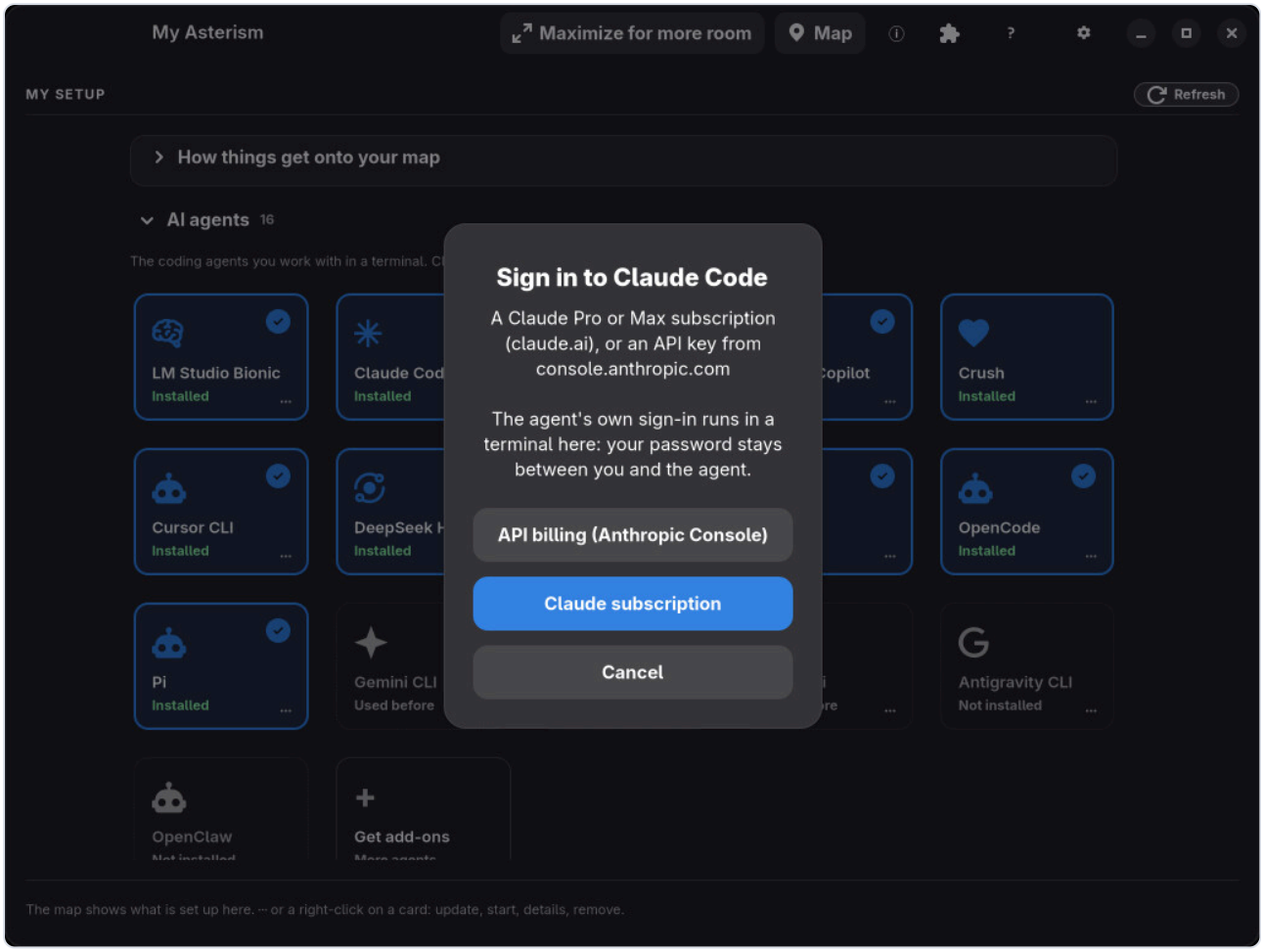
4.9 Konfiguruj…

Prawy klik na agencie lub lokalnym LLM › **Konfiguruj…** otwiera jego własną konfigurację w karcie terminala w oknie, dla tego, czego aplikacja nie ustawia sama — na przykład `hermes setup` albo własne menu Ollamy; LM Studio otwiera swoje okno. Program bez polecenia konfiguracji pokazuje swoje `--help`. Gdy się skończy, mapa sprawdza ponownie i pokazuje, co się zmieniło, a działającemu lokalnemu LLM proponuje restart.

4.10 Logowanie

Zaloguj… w menu agenta (w Konfiguracji lub na mapie) pokazuje, czego agent potrzebuje, i jego własne sposoby logowania:

- **Subskrypcja lub konto** — własne logowanie agenta działa w terminalu w oknie (na przykład `claude auth login` lub `codex login`). Twoje hasło lub logowanie w przeglądarce pozostaje między tobą a agentem; aplikacja nigdy go nie widzi. Gdy agent o to prosi, terminal mówi, co wpisać (Pi i Copilot: `/login`).
- **Klucz API** — dla agentów, które go czytają: klucz trafia do pola hasła, a potem bezpośrednio do własnego polecenia agenta (Codex przechowuje go sam) lub do pliku, który agent czyta (Gemini: `~/.gemini/.env`, czytelny tylko dla ciebie). Nigdy nie trafia do wiersza poleceń, logu ani ustawień aplikacji, a aplikacja go nie przechowuje. Rozliczanie API w Claude Code to jego własne logowanie do Anthropic Console, więc żaden klucz nie przechodzi przez aplikację.



Rysunek 7 · Logowanie do Claude Code: subskrypcja lub rozliczanie API, oba przez własne logowanie Claude Code.

Potem mapa pokazuje dostawcę online agenta i jego modele.

4.11 Klucze API w podglądzie Flutter

W podglądzie Flutter agent, który czyta klucz API, pokazuje w szczegółach **Klucz API…**. Karta mówi, dokąd trafi klucz, zanim go wpiszesz: do własnego polecenia agenta (Codex: `codex login --with-api-key`) albo do pliku, który czyta agent (Gemini: `~/.gemini/.env`, zapisany jako nowy plik czytelny tylko dla ciebie; pozostałe wiersze w nim zostają). Pole ukrywa to, co wpisujesz, a **Zapisz** staje się dostępne dopiero dla czegoś, co wygląda jak klucz. Klucz trafia do osobnego procesu roboczego przez jego standardowe wejście; nigdy nie trafia do wiersza poleceń, dziennika, kopii zapasowej ani ustawień aplikacji, a aplikacja go nie przechowuje. Gdy odpowie własne polecenie agenta, pokazany zostaje jego ostatni wiersz, bez klucza.

4.12 Aktualizowanie agentów

Gdy jest nowsza wersja zainstalowanego agenta, jego karta w Konfiguracji dostaje żółte ↑ **wersja**, a stan kolumny o niej wspomina. **Aktualizuj** uruchamia aktualizację w taki sposób, w jaki agent został zainstalowany — na przykład `mise upgrade <narzędzie>` lub `claude update` — we własnym oknie terminala, abyś mógł ją śledzić.

4.13 Aktualizacja działającego agenta w podglądzie Flutter

Gdy klikniesz **Aktualizuj** przy agencie, który właśnie działa — w karcie terminala tutaj lub we własnym terminalu — podgląd Flutter najpierw o tym powie: aktualizacja podmienia program agenta w trakcie pracy, więc jego sesje mogą się przerwać lub zachowywać dziwnie, a niektóre aktualizatory odmawiają. **Anuluj** zostawia wszystko bez zmian; pozwól agentowi skończyć i zaktualizuj później. **Zaktualizuj mimo to** uruchamia aktualizację w terminalu jak zwykle.

4.14 Agenci, których aplikacja może zainstalować

Czterech agentów można zainstalować, aktualizować i odinstalować z mapy (prawy klik na agencie › **Zainstaluj…**):

- **Goose** (`goose` , od Agentic AI Foundation, wcześniej Block), jego wydanie z GitHuba przez mise. Działa z wieloma dostawcami i z lokalnymi LLM; **Konfiguruj…** uruchamia `goose configure` .
- **DeepSeek Harness** (`dsh` , wersja zapoznawcza dla deweloperów), z npm. Może korzystać z lokalnych LLM.
- **Antigravity CLI** (`agy` , Google), z manifestu wydań Google; pobrany plik jest sprawdzany jego sumą SHA-512.
- **LM Studio Bionic** nie może być zainstalowany stąd (brak opublikowanej sumy kontrolnej); pobierz go z lmstudio.ai. Zawsze korzysta z silnika i modeli LM Studio.

4.15 Nowi agenci

Gdy pojawi się agent, którego aplikacja jeszcze nie zna, prosi raz zainstalowanego agenta (Claude Code, w przeciwnym razie Codex), aby sprawdził w sieci, gdzie ten agent trzyma swoje skille i serwery MCP. Odpowiedź jest sprawdzana przed użyciem i nic z niej nie jest nigdy wykonywane. Ustawienia › **Sprawdzaj nowe agenty** wyłącza to.

ROZDZIAŁ 5

Lokalne LLM

Lokalne środowisko LLM uruchamia modele językowe na twojej własnej karcie graficznej, więc zapytania nigdy nie opuszczają komputera. Aplikacja instaluje je, uruchamia i zatrzymuje za ciebie.

5.1 Instalacja lokalnego LLM

Prawy klik na szarym (niezainstalowanym) lokalnym LLM na mapie › **Zainstaluj…**. Wybierasz folder programu, folder modeli (proponowany jest ten, którego używają twoje inne lokalne LLM) i czy ma startować przy logowaniu. Jest instalowany tylko dla twojego użytkownika, z usługą użytkownika, więc hasło nie jest potrzebne. Każdy pobrany plik musi pasować do sumy SHA-256 opublikowanej przez jego wydanie, zanim zostanie rozpakowany lub uruchomiony.

Środowisko	Uwagi
Ollama	Oficjalne wydanie (z ROCm dla kart AMD).
llama.cpp	Wybrana kompilacja (Vulkan, ROCm, CUDA, CPU), jako router modeli: dowolny GGUF w folderze modeli, jeden załadowany naraz.
LM Studio	Nie jest instalowany stąd (brak opublikowanej sumy kontrolnej); pobierz go z lmstudio.ai. Po zainstalowaniu jest pokazywany i zarządzany.
vLLM	Dla kart NVIDIA, we własnym środowisku Pythona.
Jan	Jego AppImage, rozpakowany, z silnikiem Jana jako usługą w tle dla agentów.
KoboldCpp	Oficjalna kompilacja w jednym pliku, jako router modeli.

Tak zainstalowany lokalny LLM oferuje też **Zaktualizuj…** (poprzednia wersja zostaje jako zapas), **Folder modeli…** i **Odinstaluj…**, które usuwa program, polecenie, usługę i ustawienia, ale nigdy modele. Lokalne LLM zainstalowane przez ciebie samodzielnie są pokazywane, ale nigdy nie są zmieniane.

5.2 Instalowanie w podglądzie Flutter

W podglądzie Flutter lokalny LLM, który nie jest jeszcze zainstalowany, pokazuje **Zainstaluj** w Dostawcy i modele, a przewodnik pierwszego agenta oferuje **Zainstaluj mise**, **Zainstaluj Node.js** i **Zainstaluj** dla lokalnego LLM, którego potrzebuje jego ścieżka. Maprisma najpierw wyszukuje najnowsze wydanie i pokazuje je na karcie: jego pliki i ich rozmiar, folder programu (`~/.local/share/ai-agents-map/<nazwa>` , jeden folder na wersję), folder modeli w nim i twoją kartę graficzną. Dla llama.cpp i KoboldCpp wybierz tam wariant (Vulkan, ROCm, CUDA lub CPU); inny wariant jest ponownie wyszukiwany, zanim cokolwiek zostanie zainstalowane. Lokalny LLM z usługą użytkownika oferuje też **Uruchom teraz** i **Uruchamiaj po zalogowaniu**.

Zainstaluj pobiera pliki, sprawdza każdy z sumą kontrolną podaną w jego wydaniu i rozpakowuje je; podczas instalacji nic z pobranych plików nie jest uruchamiane (AppImage Jana rozpakowuje się sam w zamkniętej piaskownicy). Program już zainstalowany w inny sposób, istniejący plik ustawień ani istniejąca usługa nigdy nie są zastępowane: karta tego odmawia. Ollama, llama.cpp, KoboldCpp, Jan i mise instalują się w ten sposób; LM Studio i vLLM nie, bo nie ma dla nich wydania z sumą kontrolną.

Każdy krok uruchomiony z przewodnika pierwszego agenta — instalacja, logowanie, pytanie testowe, Dostawcy i modele, Połączenia — wraca przyciskiem **Wstecz** do przewodnika, odświeżonego, tak że właśnie wykonany krok jest oznaczony jako wykonany.

5.3 Uruchamianie i zatrzymywanie z agentami

Połącz raz agenta z lokalnym LLM, a potem po prostu uruchamiaj agenta — także przy zamkniętej aplikacji lub po restarcie. Jego lokalny LLM startuje razem z nim, a gdy zakończy się ostatni agent, który go używa, LLM zatrzymuje się około pięciu sekund później, więc jego pamięć graficzna znów jest wolna.

Zajmuje się tym mała usługa w tle, `ai-agents-map-providers.service`. Jest konfigurowana przy pierwszym połączeniu z lokalnym LLM i usuwana, gdy zniknie ostatnie, lub gdy wyłączysz Ustawienia › **Uruchamiaj lokalne LLM razem z agentami**.

> **UWAGA** KoboldCpp potrzebuje kilku sekund na start, dłużej niż czekają niektórzy agenci. Dlatego startuje przy logowaniu, a gdy zakończy się ostatni agent, tylko jego model jest zwalniany.

5.4 Szczegóły lokalnego LLM

Kliknij kartę lokalnego LLM w Konfiguracji albo prawym przyciskiem jego pole na mapie › **Szczegóły**. Pod kafelkami (modele, agenci, adres) znajdują się te same zwijane karty co u agentów: **Użycie i tokeny** — co twoi agenci przez niego wysłali w ostatnich dniach, według agenta i modelu, z własnych liczb sesji agentów i dopasowane po nazwie modelu —, potem Stan, Zarządzanie ryzykiem, Co znaczą te słowa i Konfiguracja.

5.5 Wydajność

Lokalny LLM, którego ustawienia aplikacja zna (llama.cpp, Ollama zainstalowana przez aplikację), ma na swojej stronie szczegółów kartę **Wydajność**:

- **Wariant**, **Stosy GPU** i **Pamięć wideo**: na czym działa — ROCm, Vulkan, CUDA lub procesor (dla llama.cpp odczytane z jego własnej listy urządzeń, także dla wariantu zbudowanego samodzielnie), które z tych stosów ma ten komputer i pamięć karty.
- **Kontekst na model**: rozmiar pliku każdego modelu, ustawiony teraz kontekst, kontekst, jaki karta dla niego zmieści — jego plik plus pamięć podręczna kontekstu, odczytana z nagłówka modelu — oraz to, do jakiego kontekstu był trenowany. **!** oznacza kontekst większy, niż karta zmieści (model by się nie załadował albo przelałby się do RAM), ↑ taki, który karta mogłaby powiększyć; **Ustaw na …** go zmienia.
- **Zapytaj …**: lokalny pomocnik (lub agent online) przegląda te fakty — tylko nazwy i liczby — i proponuje zmiany: ustawienia takie jak flash attention, typ pamięci podręcznej kontekstu, warstwy

GPU, równoległe żądania i rozmiary partii; konteksty; inny wariant, gdy wyraźnie by pomógł. Aplikacja zostawia tylko propozycje, za którymi może stanąć (znane ustawienie z dozwoloną wartością, kontekst, który karta zmieści), i pokazuje każdą z uzasadnieniem. **Zastosuj** zapisuje ją z kopią zapasową; komunikat oferuje **Cofnij** albo **Uruchom ponownie teraz**, gdy zmiana zadziała dopiero po restarcie. Inny wariant jest tylko proponowany: zainstaluj go z menu lokalnego LLM.

W Ollama ustawienia dotyczą wszystkich modeli naraz (plik środowiska jej serwera).

5.6 Wydajność w podglądzie Flutter

W podglądzie Flutter lokalny LLM z ustawieniami, które aplikacja może dostroić (llama.cpp, Ollama), ma **Wydajność** w **Dostawcy i modele**. Pokazuje wersję, w jakiej działa, stosy GPU na tym komputerze i pamięć wideo, a potem **Kontekst na model**: rozmiar każdego modelu, jego obecny kontekst, ile karta dla niego zmieści i na ile był trenowany. Model z większym kontekstem, niż mieści karta, jest oznaczony, tak samo jak model, dla którego karta ma jeszcze miejsce; w llama.cpp **Ustaw na …** go ustawia. **Ustawienia dla wszystkich modeli** wymienia ustawienia silnika wraz z tym, co robią; ustawienie z listą wartości można tam zmienić. Każda zmiana jest najpierw pokazana i zapisana z kopią zapasową, więc **Cofnij** ją przywraca; działa po ponownym uruchomieniu lokalnego LLM. W **Wskazówki od agenta** przycisk **Zapytaj …** każe agentowi do kontroli AI przejrzeć te fakty (tylko nazwy i liczby, tylko przez sieć): proponuje ustawienia, konteksty, inną wersję lub uwagi, każdą z uzasadnieniem. Aplikacja zostawia tylko to, za czym może stanąć — dozwoloną wartość, kontekst, który mieści karta, wersję, którą ma lokalny LLM — a **Zastosuj** przechodzi przez ten sam podgląd. Tokeny są agenta; przebieg widać w Smart Check.

5.7 Pamięć GPU i zwalnianie pamięci GPU

W karcie **GPU** nad mapą **Pamięć GPU** pokazuje, ile jest zajęte, z podziałem na lokalne LLM, pulpit i aplikacje oraz wolne; **Aktywność GPU** pokazuje obciążenie, temperaturę i moc. Pole każdego lokalnego LLM pokazuje, ile pamięci zajmuje, a załadowane modele dostają zieloną kropkę.

Prawy klik na lokalnym LLM daje **Uruchom**, **Zatrzymaj**, start przy logowaniu i **Zwolnij pamięć GPU**: jego modele są zwalniane, działa dalej i ładuje model ponownie, gdy zostanie o to poproszony.

Gdy zakończy się ostatni agent, który używał lokalnego LLM, a LLM wciąż trzyma swoje modele, komunikat pyta, czy oddać pamięć: **Zwolnij** wyładowuje je, **Zachowaj** zostawia je załadowane.

Lokalny LLM, który zajmuje pamięć GPU, dostaje na swoim polu czerwony przycisk zatrzymania. Najpierw pyta: **Zwolnij pamięć GPU** wyładowuje jego modele; **Zatrzymaj** go kończy. W LM Studio przycisk to **Zatrzymaj serwer**: kończy się tylko jego serwer, a aplikacja zostaje otwarta z załadowanymi modelami. Model, który lokalny LLM ma załadowany, ale którego nie używa żaden agent, jest z nim połączony ciągłą jasną linią.

LM Studio może być otwarte bez swojego serwera. Jego pole pokazuje wtedy *serwer wyłączony* z żółtą kropką, agenci nie mogą się z nim połączyć, a jego menu oferuje **Uruchom serwer** (`lms server start`).

5.8 Modele jednocześnie

llama.cpp, LM Studio i Ollama mogą trzymać załadowany więcej niż jeden model. Gdy łączysz agentów z różnymi modelami jednego lokalnego LLM, aplikacja ustawia, ile modeli trzyma on załadowanych jednocześnie — tyle, ile używają jego agenci — z tym samym podglądem, kopią zapasową i Cofnij co połączenie, i proponuje restart, jeśli działa, bo odczytuje tę liczbę przy starcie. Tam, gdzie ta liczba nie należy do aplikacji (instalacja systemowa albo twoje własne polecenie startowe), notatka mówi, gdzie ją ustawić.

Gdy modele razem nie mieszczą się w pamięci GPU, ostrzeżenie o tym mówi: lokalny LLM je wtedy przełącza, co jest wolne, albo część modelu działa na procesorze. Uruchomienie agenta sprawdza to samo: jeśli załadowanie jego modelu potrzebuje więcej pamięci GPU, niż jest wolne, zostaniesz najpierw zapytany, z tym, co teraz zajmuje pamięć — **Zwolnij pamięć GPU, potem uruchom**, **Uruchom mimo to** albo **Anuluj**.

5.9 Modele jednocześnie w podglądzie Flutter

W podglądzie Flutter, po zastosowaniu zmiany lokalnego LLM lub modelu agenta, aplikacja sprawdza, ile różnych modeli używają teraz agenci każdego lokalnego LLM. Tam, gdzie może ustawić tę liczbę — pliki ustawień llama.cpp i Ollamy z instalacji samej aplikacji, ustawienie „jeden lub wiele” LM Studio — drugi **Podgląd** pokazuje zmianę wraz z jej znaczeniem i ewentualnymi ostrzeżeniami; **Zastosuj** zapisuje ją z kopią zapasową, a **Cofnij** ją wycofuje. Liczba ustalona w twoim własnym skrypcie startowym zostaje bez zmian. Gdy modeli jest więcej, niż pozostaje załadowanych, albo razem są za duże dla karty graficznej, mówi to też stan kolumny Dostawcy.

5.10 Używany przez inne aplikacje

Z lokalnych LLM korzystają nie tylko agenci. Wtyczka taka jak Video Trans, aplikacja czatu czy twój własny skrypt też mogą wysyłać zapytania. Aplikacja to zauważa i pokazuje:



Rysunek 8 · Program spoza mapy (tu `curl`) generujący z llama.cpp: cyjanowa linia i pole (motyw Tokyo Night). Własna ścieżka Claude Code do opus pozostaje w kolorze akcentu.

- **Na mapie** linia od lokalnego LLM do modelu, którym generuje, staje się gruba i ma własny kolor, biegnie wzdłuż niej światło, a pole modelu dostaje kropkę w tym kolorze i mówi, kto go używa, na przykład *w użyciu · Video Trans*.
- **Menu prawego przycisku lokalnego LLM** i jego strona ⓘ **Szczegóły** mówią *Używany przez Video Trans · teraz* lub ile sekund temu był ostatnio używany.
- Legenda ⓘ pokazuje oba znaczniki.

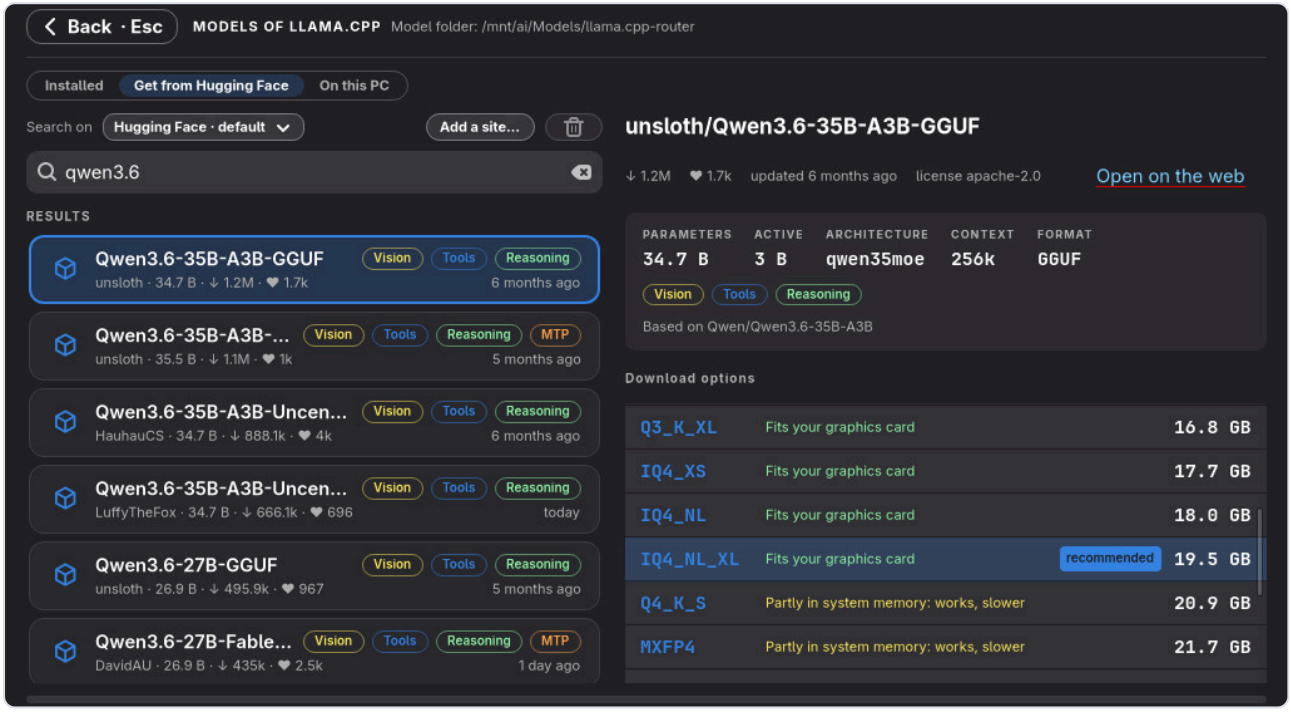
Aplikacja sprawdza, które z twoich własnych programów mają otwarte połączenie z lokalnym LLM. Nigdy nie czyta, co wysyłają. Wtyczka jest pokazywana pod własną nazwą, agent pod swoją, a wszystko inne pod nazwą programu. Sama aplikacja i LLM Monitor, które tylko czytają stan, się nie liczą.

> **UWAGA** Światło widać tylko wtedy, gdy program generuje, i gaśnie około dziesięciu sekund po jego ostatnim zapytaniu. Program otwarty, ale bezczynny — Video Trans bez mowy do tłumaczenia — niczego nie pokazuje. Program pozostaje w menu przez pół minuty po ostatnim zapytaniu.

ROZDZIAŁ 6

Modele

Prawy klik na zainstalowanym lokalnym LLM › **Modele…** albo kliknij **Dodaj model** w **Konfiguracji**. Strona ma listę po lewej i wybrany model po prawej, w trzech kartach.



Rysunek 9 · Pobieranie z Hugging Face: co potrafi model i jaki rozmiar zmieści się w twojej karcie graficznej.

6.1 Pobieranie z Hugging Face

Przeglądaj popularne modele czatu GGUF lub szukaj. Każdy wynik pokazuje autora, rozmiar, pobrania, polubienia, wiek i to, co potrafi: **Obrazy**, **Narzędzia**, **Rozumowanie**, **MTP**. Strona modelu pokazuje parametry, architekturę, kontekst, licencję i kartę modelu. **Co znaczy nazwa** objaśnia części nazwy, takie jak *35B-A3B* czy *Q4_K_M*.

Opcje pobierania wypisują każdy rozmiar z tym, jak się mieści: w karcie graficznej, częściowo w pamięci RAM (działa, ale wolniej) lub prawdopodobnie za duży. Najlepszy wybór jest już zaznaczony; **Pobierz** zapisuje go w folderze modeli. Dla Ollamy, która pobiera po nazwie rozmiaru, oferowane są tylko rozmiary, które może pobrać: plik podzielony, plik w podfolderze lub taki, którego Hugging Face odmawia Ollamie, jest przyciemniony, z podanym powodem, i nigdy nie jest polecany.

Pobieranie odbywa się w tle i pojawia się w pasku u góry strony głównej. Gdy połączenie zostanie przerwane, próbuje ponownie i wznawia. Jeśli aplikację zamknięto w trakcie pobierania, jest ono wypisane jako przerwane, z **Wznów** lub **Odrzuć**.

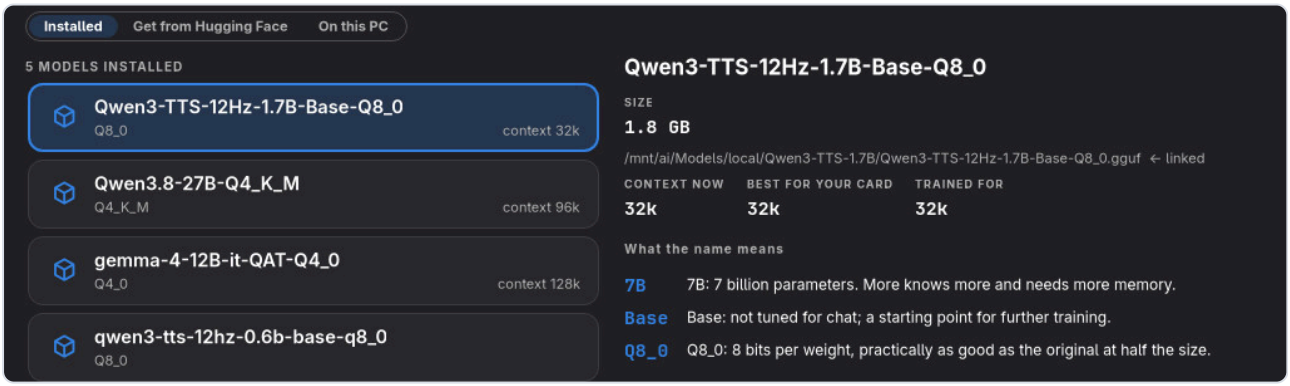
6.2 Przerwane pobierania w podglądzie Flutter

W podglądzie Flutter pobieranie modelu lub pull Ollamy, którego połączenie się zerwie, jest ponawiane po 5, 15, 30, 60 i 120 sekundach i trwa dalej od miejsca, w którym stanęło. Jeśli aplikacja zakończy się wcześniej, następne uruchomienie pokazuje baner z każdym przerwanym pobieraniem, miejscem, w którym stanęło, **Wznów** (kontynuuje od już pobranej części) i **Odrzuć**. Zapisane jest tylko, jak je uruchomić ponownie, w `~/.local/state/maprisma/jobs.json`, czytelnym tylko dla ciebie; pobieranie, które wciąż działa gdzie indziej, nie zostanie uruchomione drugi raz.

6.3 Zapytaj agenta, który model pasuje

Nie wiesz, który model wybrać? Gdy skonfigurowany jest agent do kontroli AI (Ustawienia › **Agent do kontroli AI**), karta **Zapytaj agenta** pod wyszukiwarką przyjmuje pytanie zadane własnymi słowami — na przykład *model do programowania z długim kontekstem, który zmieści się w mojej karcie graficznej*. **Pytaj** (lub `Ctrl`+`Enter`) wysyła je, razem z kartą graficzną, jej pamięcią, pamięcią komputera i modelami, które już masz, do tego agenta, który szuka w sieci; podczas pracy widać animowany wskaźnik i czas, który już upłynął. Po minucie lub dwóch karta pokazuje jego krótkie podsumowanie, a proponowane modele pojawiają się poniżej na liście wyników pod nagłówkiem *Propozycje: …*: każdy znaleziony w Hugging Face, z rozmiarem i powodem, dla którego pasuje; kliknięcie otwiera jego stronę i opcje pobierania jak przy każdym wyniku wyszukiwania. Propozycja, której nie ma w Hugging Face, jest wymieniona pod nagłówkiem. Proponuje do 10 modeli; **Propozycje** po prawej stronie wiersza agenta ustawia inną liczbę (5, 10, 15 lub 20). Zużywa tokeny agenta; pytanie nie jest zapisywane, a strona każdego lokalnego LLM pyta osobno.

6.4 Zainstalowane



Rysunek 10 · Zainstalowane: kontekst obecny, najlepszy dla twojej karty i treningowy; co znaczy nazwa; Usuń.

Każdy model pokazuje rozmiar, plik i kontekst — *obecny*, *najlepszy dla twojej karty* i *trenowany na*. **Usuń** najpierw pyta i przenosi pliki do kosza, skąd można je przywrócić. Model, który jest tylko dowiązaniem, traci tylko dowiązanie.

6.5 Rozmiar kontekstu

Kontekst to ilość tekstu, którą model może mieć jednocześnie na uwadze. Aplikacja oblicza z samego pliku modelu największy kontekst, który zmieści się w twojej karcie graficznej. **Ustaw kontekst** zapisuje go w ustawieniach lokalnego LLM, z kopią zapasową i Cofnij. Agenci potrzebują dość dużego

kontekstu; gdy łączysz agenta z modelem LM Studio bez ustawionego kontekstu, najlepszy jest ustawiany w tym samym kroku.

6.6 Na tym komputerze

Wypisuje pliki GGUF, których lokalny LLM jeszcze nie czyta — w jego folderze modeli, w folderach innych lokalnych LLM, w `~/Downloads` i w `~/Models`. **Dodaj** robi z jednego z nich jego model bez drugiej kopii (przez dowiązanie), z wyjątkiem Ollamy, która importuje własną kopię.

6.7 Przenoszenie i usuwanie modeli

Prawy klik na modelu llama.cpp, KoboldCpp lub LM Studio › **Przenieś do…** albo przeciągnij pole modelu na folder innego lokalnego LLM. Najpierw widzisz, ile zostanie przeniesione i który agent używa modelu. Cofnij przenosi pliki z powrotem. **Usuń…** najpierw pyta i przenosi pliki do kosza.

6.8 Jeden folder modeli

Ustawienia › **Folder modeli** ustala jeden folder na modele wszystkich lokalnych LLM. **Uporządkuj modele…** pokazuje z góry, jak pliki zostałyby posortowane (`<folder>/<wydawca>/<model>/`), i nic się nie zmienia, dopóki nie naciśniesz **Uporządkuj**. Z opcją **Stare ścieżki nadal działają** (domyślnie włączoną) w każdym starym miejscu zostaje dowiązanie. **Cofnij ostatnie porządki** przywraca wszystko.

> **OSTROŻNIE** Przy przenoszeniu na inny dysk modele są najpierw kopiowane, a oryginały usuwane dopiero po ukończeniu kopii. Upewnij się, że w miejscu docelowym jest dość wolnego miejsca; w przeciwnym razie aplikacja odmówi przeniesienia.

ROZDZIAŁ 7

Łączenie przez rysowanie

Naciśnij pole na mapie i przeciągnij do innego, aby je połączyć. Każda zmiana pokazuje przed zapisem dokładnie, które wiersze którego pliku się zmienią, tworzy kopię zapasową i można ją cofnąć z paska, który się pojawi.

7.1 Agent ↔ lokalny LLM

Przeciągnij od agenta do lokalnego LLM lub odwrotnie. Wybierz model i czy agent ma go od razu używać. Aby zmienić połączenie, zaznacz jego agenta (lub kliknij linię): każda linia, którą można przenieść lub usunąć, ma na końcu **pomarańczowy uchwyt**. Przeciągnij uchwyt do innego lokalnego LLM — lub w puste miejsce, aby ją usunąć; po najechaniu na uchwyt widać, co da jego przeciągnięcie. Gdy agent jest zaznaczony, kliknięcie modelu połączonego lokalnego LLM czyni ten model modelem agenta.

Obsługiwani agenci: Hermes, Pi, Oh My Pi, OpenCode, OpenClaw i DeepSeek Harness.

7.2 Agent ↔ serwer MCP

Przeciągnij od agenta do serwera MCP. Ustawienia serwera są przenoszone do własnego formatu tego agenta. Obsługiwani: Claude Code, Codex, Copilot, Crush, Cursor, Gemini, Goose, Grok, Hermes, Muse, Oh My Pi, OpenClaw, OpenCode i Pi. Pliki ustawień, które mogą zawierać klucze, są zawsze zapisywane tak, aby tylko ty mógł je czytać.

7.3 Agent ↔ folder skilli

Przeciągnij od agenta do folderu skilli. Pi i Hermes dostają folder w swoich ustawieniach; każdy inny agent dostaje dowiązanie do każdego skilla we własnym folderze skilli. Folderu, który agent czyta sam z siebie (większość czyta `~/.agents/skills`), nie da się odebrać tylko temu agentowi.

7.4 Znalezione na tym komputerze

Przy każdym otwarciu okna (i po **Odśwież**) aplikacja szuka na tym komputerze serwerów MCP i folderów skilli, niczego nie uruchamiając: programów oferujących serwer MCP w `~/.local/bin`, `~/Applications`, `/opt` i w folderach dodanych w Konfiguracji oraz folderów skilli, które przyniosły aplikacje albo które leżą w dodanym folderze. To, czego żaden agent jeszcze nie używa, jest na mapie jako wyblakłe pole bez linii. Nic nie łączy się samo.

1. Przeciągnij od agenta do wyblakłego pola.
2. Strona połączenia pokazuje, jak serwer zostanie uruchomiony — poleceniem, które podaje jego własny tekst pomocy, na przykład z folderem, który może czytać i zapisywać — i dokładnie, co zmienia się w ustawieniach agenta.

3. **Połącz** zapisuje to. Pole przestaje być wyblakłe, a aplikacja od razu sprawdza, czy serwer odpowiada (zob. Gdy połączenie się nie uda).

Serwer MCP aplikacji działa samodzielnie: okno aplikacji nie musi być otwarte. Programów w AppImage, który nie został rozpakowany, nie ma na mapie; **Dodaj serwer MCP** › **Na tym komputerze** najpierw je rozpakowuje.

> **WSKAZÓWKA** Trzymaj aplikacje z serwerem MCP i własne skille w osobnym folderze każde — na przykład na dysku, który dzieli kilka systemów — i dodaj te foldery w Konfiguracji. Na innym komputerze dodaj tam te same foldery, a wszystko, co w nich jest, znajdzie się też na jego mapie.

7.5 Lokalne połączenia w podglądzie Flutter

Otwórz Połączenia i Dodaj serwer MCP. Wpisz nazwę, wybierz stdio, HTTP lub SSE oraz zainstalowane agenty. Dla stdio podaj program oddzielnie, a argumenty po jednym w wierszu; spacje pozostają częścią argumentu. Dodaj zmienne lub nagłówki jako pary nazwa/wartość; wartości są ukryte. Podgląd pokazuje odbiorców i liczby zmian bez wartości konfiguracji. Zastąpienie definicji o tej samej nazwie jest ujawniane przed zatwierdzeniem. Zastosuj używa kopii zapasowej i Cofnij. Maprisma nie uruchamia ani nie testuje tu serwera; agenty mogą go uruchomić po odczytaniu konfiguracji.

Dodaj umiejętności wybiera istniejący folder zawierający foldery z SKILL.md. Wybierz folder nadrzędny i agenty, a następnie sprawdź dowiązania lub ustawienia. Oryginały pozostają; dowiązania uwzględniają późniejsze zmiany. Umiejętności mogą zawierać skrypty wykonywane przez agenty. Natywne foldery już odczytywane nie wymagają nowych dowiązań. Sprawdzanych jest 200 wpisów na folder, jeden poziom kategorii i łącznie 1 000 wpisów; w razie potrzeby wybierz mniejszy folder. Podgląd obsługuje 32 agenty i 200 dowiązań. Cofnij zachowuje późniejsze zmiany. Katalogi internetowe, pobieranie i testy narzędzi MCP nie należą do tych lokalnych formularzy.

7.6 Znalezione na tym komputerze w podglądzie Flutter

W podglądzie Flutter aplikacja przy starcie szuka w tle, niczego nie uruchamiając, programów na tym komputerze, które oferują serwer MCP (ich pliki zawierają słowa właściwe protokołowi; sposób uruchomienia pochodzi z ich własnego tekstu użycia), oraz zestawów skilli, które przyniosły aplikacje. Duże programy są czytane raz i zapamiętywane. Stan kolumny MCP pokazuje **Nowe na tym komputerze** dla tego, czego jeszcze nie pokazano; **Dodaj serwer MCP…** lub **Dodaj skille…**, albo **Znalezione na tym komputerze** w Połączeniach, wyświetla je, nowe z ★. **Dodaj…** otwiera formularz wypełniony nazwą i poleceniem startu albo folderem, do sprawdzenia i zmiany przed podglądem. Samo wyszukiwanie nigdy nie uruchamia AppImage. **Zajrzyj do środka…** rozpakowuje tylko programy każdego z nich, zamknięte przez bubblewrap (bez sieci, bez folderu domowego, zapisywalny tylko folder tymczasowy), czyta je i znowu usuwa; to, co znaleziono, jest zapamiętywane. Dodanie serwera z AppImage najpierw pyta, czy rozpakować cały AppImage do `~/Applications/<app>-<version>/` (**Rozpakuj**), w ten sam sposób. Nowszy AppImage aplikacji, do której już zajrzano lub którą rozpakowano, jest tylko wymieniany (**Znaleziono nową wersję**): plik o nowszej nazwie nie dowodzi prawdziwej aktualizacji, więc zagląda się do niego jak do każdego innego.

Gdy aplikacja zostanie znaleziona w nowszej wersji, a agenci, lista serwerów MCP lub link w `~/.local/bin` wciąż uruchamiają starszą, lista pokazuje ją z wersjami i tym, kto jej używa. **Zaktualizuj…** (po rozpakowaniu nowszego AppImage, jeśli to on) pokazuje w podglądzie jedną zmianę dla wszystkiego: każdy wpis dostaje nowy program (argumenty wskazujące stary folder przenoszą się razem z nim), linki wskazują nowy folder, a stary folder trafia do kopii zapasowej. Zostaje tam 7 dni; Cofnij przywraca wszystko.

7.7 Gdy połączenie się nie uda

Zaraz po zapisaniu połączenia aplikacja sprawdza, czy będzie działać: czy lokalny LLM jest zainstalowany i oferuje model, czy serwer MCP odpowiada, czy dowiązania skilli są całe. Nieudane połączenie zostaje na mapie jako **czerwona linia przerywana** z **!**; takie z samymi ostrzeżeniami jest **żółte przerywane**. Kliknij linię lub jej **!**, aby zobaczyć wskazówki:

- **Testuj ponownie** sprawdza jeszcze raz; jeśli przejdzie, linia wraca do normy.
- **Spróbuj ponownie…** otwiera stronę łączenia dla połączenia, które nigdy nie zostało zapisane.
- **Usuń połączenie** je usuwa.

ROZDZIAŁ 8

Serwery MCP

Serwery MCP dają agentom dodatkowe narzędzia. **+ Serwer MCP** na mapie lub pod kartami otwiera stronę dodawania: lista po lewej, strona wybranego serwera po prawej.

8.1 Skąd pochodzą serwery

Karta	Zawartość
Polecane	Serwery referencyjne projektu MCP (Filesystem, Fetch, Git, Memory, Sequential Thinking, Time, Everything) i oficjalne serwery znanych firm (GitHub, Playwright, Chrome DevTools, Brave Search, Context7), pogrupowane według przeznaczenia i przypięte do najnowszej wersji.
Rejestr MCP	Wyszukiwanie w oficjalnym rejestrze MCP. Każdy wynik podaje wydawcę tak, jak zweryfikował go rejestr. Wyszukiwanie może trwać dziesięć sekund lub dłużej. Wyniki przychodzą po 20; wiersz pod listą przewija je stronami i pokazuje, gdzie jesteś, na przykład **‹ Poprzednie 20** · 21 – 40 / 200 · **Następne 20 ›** (z *40+*, gdy rejestr nie podaje, ile ich jest).
Na tym komputerze	Programy na tym komputerze, które oferują serwer MCP, znalezione przez czytanie ich plików — nic nie jest uruchamiane. Do wnętrza AppImage zaglądamy tylko po kliknięciu **Zajrzyj do środka…**; są rozpakowywane w zamknięciu przez bubblewrap (bez sieci, bez folderu domowego), więc `bwrap` musi być zainstalowany.
Ręcznie	Wiersz poleceń lub adres internetowy z nagłówkami.

Nie wiesz, jakiego serwera potrzebujesz? Na karcie **Rejestr MCP** karta **Zapytaj agenta** pod wyszukiwarką przyjmuje prośbę zapisaną własnymi słowami — na przykład *czytać i pisać moje strony w Notion*. **Pytaj** (lub `Ctrl`+`Enter`) sprawia, że agent do kontroli AI zamienia ją na kilka słów wyszukiwania; aplikacja przeszukuje nimi pobraną listę rejestru, tak jak zwykłe wyszukiwanie, a agent wybiera z wyników pasujące serwery, każdy z uzasadnieniem. Nie szuka w sieci, widzi tylko Twoją prośbę i publiczne teksty rejestru i może wybrać tylko serwery, które naprawdę są na liście. Podczas pracy widać animowany wskaźnik i czas, który upłynął; po kilku sekundach do minuty propozycje pojawiają się na liście wyników, do 10 (**Propozycje** po prawej stronie wiersza agenta: 5, 10, 15 lub 20). Wyszukiwanie wpisane później je zastępuje.

Na tym komputerze szuka też w folderach dodanych przyciskiem **Dodaj folder…** w Konfiguracji › **Lokalne serwery MCP**: w każdym folderze i w folderach aplikacji, które zawiera, takich jak `photocraft-0.16.0/usr/bin`.

8.2 Konfiguracja serwera

Strona serwera mówi, kto go stworzył i jak dobrze jest to zweryfikowane, co robi (z rzeczami, o które możesz poprosić agenta), jak działa — jako program przez `npx`, `uvx` lub Docker albo jako usługa sieciowa — i czego potrzebuje, na przykład folderu lub klucza API z linkiem, gdzie go zdobyć. Poniżej

Skonfiguruj:

1. Wypełnij pola, których potrzebuje serwer.
2. Zaznacz agentów, którzy mają go dostać.
3. Kliknij **Testuj**, aby zapytać serwer o jego narzędzia.
4. Kliknij **Dodaj**: wszyscy zaznaczeni agenci są zapisywani naraz, z podglądem i Cofnij.

> **UWAGA** Wartości tajne, takie jak klucze API, są przekazywane w środowisku lub nagłówkach, nigdy w wierszu poleceń, gdzie mogliby je odczytać inni użytkownicy komputera. Serwer, który potrzebowałby sekretu w wierszu poleceń, trzeba skonfigurować ręcznie.

8.3 Serwery MCP w podglądzie Flutter

W podglądzie Flutter **Znajdź serwery MCP…** w **Połączenia** otwiera karty **Polecane** (przy otwarciu przypięte do najnowszej wersji) i **Rejestr MCP** (po 20 wyników z **‹ Poprzednie 20** i **Następne 20 ›**; lista rejestru pobiera się w tle, by wyszukiwanie było szybsze). Strona serwera mówi, kto go zrobił, co robi, czego potrzebuje i **Jak działa**; sposób, który może działać na tym komputerze, jest oznaczony jako **zalecany**, a brakujący `npx`, `uvx` lub Docker jest wymieniony. Wypełnij potrzebne pola (tajne są ukryte), sprawdź **Nazwa**, zaznacz agentów i kliknij **Podgląd**: podgląd, **Zastosuj** i **Cofnij** są takie jak przy zmianie połączeń. Do usługi trafiają tylko wpisane wartości; samo polecenie pochodzi z wpisu serwera i jest tam sprawdzane ponownie. Sekret, który trafiłby do wiersza poleceń, jest odrzucany; taki serwer trzeba skonfigurować ręcznie.

Testuj uruchamia raz wybrany sposób (albo pyta jego adres) i wypisuje narzędzia, które oferuje, zanim cokolwiek zostanie zapisane. Już skonfigurowany serwer pokazuje **Narzędzia** w swojej karcie w **Połączenia**: ostatnią odpowiedź z datą oraz **Zapytaj serwer** (lub **Zapytaj ponownie**), co uruchamia go raz tak, jak robi to jego pierwszy agent, i potem zatrzymuje. Są tam też jego **Prompty**. Odpowiedź zostaje na następny raz; klucze z jego ustawień trafiają prywatnie do pytającego procesu i nigdy nie są pokazywane.

8.4 Zarządzanie serwerami

ⓘ **Szczegóły** na karcie serwera MCP pokazują plik ustawień każdego agenta, polecenie lub adres i, na żądanie, narzędzia serwera; wartości kluczy nigdy nie są pokazywane. **Usuń** odbiera go wszystkim agentom. Serwery zostają na liście, gdy żaden agent już ich nie używa, aby linia mogła je ponownie połączyć.

Gdy w `~/Applications` pojawi się nowsza wersja aplikacji udostępniającej serwer MCP, mówi o tym wskazówka. **Zaktualizuj…** rozpakowuje nową wersję, kieruje do niej każdego agenta i zachowuje starą przez siedem dni jako kopię zapasową. **Sprawdź aktualizacje** (w **Na tym komputerze**) robi to po twoim kliknięciu dla wszystkich takich aplikacji naraz: zagląda do nowych wersji, proponuje wszystkie aktualizacje w jednym podglądzie z kopią zapasową i Cofnij, a potem testuje każdy zaktualizowany serwer; wskazówka wymienia każdy, który nie odpowiada.

ROZDZIAŁ 9

Skille

Skill to folder z instrukcjami, które uczą agenta określonego zadania. **+ Skille** otwiera stronę zbudowaną tak jak strona MCP.

9.1 Znajdowanie skilli

- **Na tym komputerze**: foldery skilli przyniesione przez aplikacje, których żaden agent jeszcze nie czyta.
- **Z sieci**: wyszukiwanie w katalogu skills.sh, na innej stronie z tym samym API lub w repozytorium GitHub ze skillami. Skill jest pobierany z jego repozytorium GitHub do wybranego folderu, domyślnie `~/.agents/skills`.

> **OSTROŻNIE** Skille mogą zawierać skrypty, które agent może później uruchomić. Strona pokazuje repozytorium i wypisuje skrypty, zanim cokolwiek zostanie pobrane — przejrzyj je.

Na tym komputerze szuka też w folderach dodanych przyciskiem **Dodaj folder...** w Konfiguracji › **Skille**.

9.2 Skille z sieci w podglądzie Flutter

W podglądzie Flutter **Znajdź skille w sieci...** w **Połączenia** szuka w skills.sh. Wynik podaje swoje repozytorium GitHub i ile razy go zainstalowano; wybranie go pobiera pliki skilla — najwyżej 150 plików i 30 MB, bez dowiązań, nic spoza jego folderu — i żadnego z nich nie uruchamia. Karta pokazuje opis, liczbę plików i skrypty, które agent mógłby później uruchomić. Wybierz folder, do którego trafi (**Do**: wspólny `~/.agents/skills` lub jeden z twoich folderów skilli), zaznacz agentów w **Także dla** i kliknij **Podgląd**: nowe pliki są wymienione, a **Zastosuj** zapisuje je z kopią zapasową, więc **Cofnij** znów je usuwa. Skill, którego folder już istnieje, nie zostaje nadpisany. Do tego czasu pobrane pliki czekają w prywatnym folderze; każdy jest porównywany z tym, co pokazał podgląd, zanim zostanie zapisany.

9.3 Foldery skilli

Każda karta folderu skilli pokazuje jego skille i agentów, którzy go czytają. Folder udostępniany przez dowiązania pojawia się raz, tam, gdzie naprawdę jest. ⓘ **Szczegóły** wypisują każdy skill z opisem i tym, jak każdy agent dostaje folder.

ROZDZIAŁ 10

Ustawienia i pomoc

10.1 Motywy i światła mapy Flutter

We Flutterze Ustawienia oferują System, Jasny i Ciemny z małymi podglądami. System korzysta z wyglądu i palety pulpitu; Jasny i Ciemny wybierają stałą jasność z dopasowanymi tłami i czytelnymi kolorami. Wybór jest zapisywany i dotyczy również kart zadań. Preferencja wyglądu GTK pozostaje oddzielna. Przy opcji System nowy motyw pulpitu pojawia się w ciągu dwóch sekund, także gdy aplikacja jest zajęta pobieraniem, instalacją lub zmianą.

Liczba klatek świateł mapy oferuje 20 kl./s (domyślnie), 30 i 60. To maksimum dla animowanych świateł, a nie interwał automatycznego skanowania. Prędkość ruchu pozostaje taka sama; wyższa częstotliwość dodaje klatki pośrednie. Faktyczna częstotliwość zależy od ekranu i dostępnego czasu renderowania. Wyłączone światła, ograniczony ruch, brak fokusu okna lub ukryta mapa nadal zatrzymują animację. Wybór jest zapisywany.

Ustawienia oferują też **Uruchamiaj lokalne LLM razem z agentami**: gdy uruchamia się agent połączony z lokalnym LLM, uruchamia się też jego LLM, a gdy zamyka się ostatni taki agent, LLM się zatrzymuje i jego pamięć graficzna znów jest wolna (robi to usługa użytkownika, także przy zamkniętym oknie). **Agent do kontroli AI** wybiera, który zainstalowany agent jest proponowany jako pierwszy do pomocy AI; zużywa na to swoje tokeny.

10.2 Tryb okna i okno wyskakujące w Omarchy

Ustawienia › **Okno** wybiera **Automatycznie** (domyślnie: zwykłe kafelkowane okno na każdym pulpicie, z ukrytym paskiem tytułu GTK w Omarchy), **Okno wyskakujące pod paskiem** (pływające okno w prawym górnym rogu pod paskiem w Omarchy) lub **Zwykłe okno**; zmiana obowiązuje przy następnym otwarciu okna. W trybie okna wyskakującego `Esc` na stronie głównej lub kliknięcie obok je zamyka; ponowne uruchomienie aplikacji (lub ikona na pasku, która wywołuje `maprisma --toggle`) przywraca je na miejsce, ze wszystkim tak, jak zostało. Okno wyboru pliku aplikacji go nie zamyka.

10.3 Dane o użyciu i narożniki z Omarchy w podglądzie Flutter

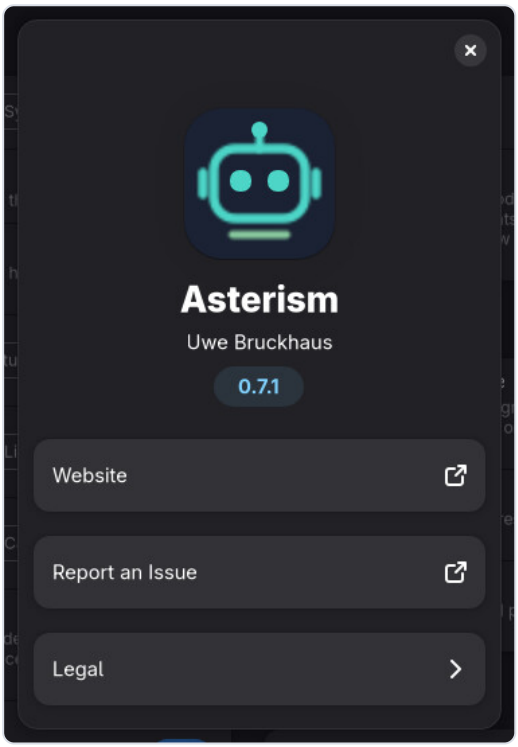
Gdy w podglądzie Flutter otwierają się szczegóły agenta, którego użycie zbiera Omarchy (plan, limity i kiedy się odnawiają), aplikacja w tle prosi Omarchy o świeże dane, jeśli ostatnie są starsze niż dziesięć minut lub starsze niż ostatnie logowanie agenta, a potem je pokazuje; najwyżej raz na dziesięć minut na agenta. Ustawienia › **Narożniki** ustala narożniki kart, okien dialogowych i pól mapy: **Jak okna** podąża za zaokrągleniem okien Hyprland (z liczbą pikseli), **Zaokrąglone** lub **Proste**.

10.4 Ustawienia

Koło zębate (`Ctrl`+`,`) otwiera ustawienia. `Backspace` lub `Esc` wraca.

Ustawienie	Co robi
Język	Język systemu albo English, Deutsch, Français, Español, Italiano, Português, Nederlands, Polski.
Wygląd OMARCHY	Kolory motywu Omarchy lub wygląd Adwaita z GNOME z jasnym/ciemnym trybem i kolorem akcentu systemu.
Animacje mapy	Światło biegnie wzdłuż linii mapy: **Agenci i inne programy** (linia pracującego agenta i linia lokalnego LLM do modelu, którego używa inny program), **Tylko moi agenci** lub **Wyłączone**. Mała animowana mapa pod ustawieniem pokazuje, co porusza każdy wybór.
Pole statystyk użycia	Liczby wzdłuż linii pracującego agenta (zob. Praca z mapą): włączone lub nie, które z **Tokeny na sekundę**, **Rozmiar kontekstu** i **Użyty kontekst** pokazuje oraz **Rozmiar tekstu** (domyślnie 7 pt).
Podagenci	Dla strony Podagenci: jak długo zostają zakończeni podagenci (domyślnie **30 minut**; 10, 30, 60 lub 120) i ilu najwyżej (domyślnie **10**; 5, 10, 20 lub 40). Pracujący są zawsze pokazywani.
Koszty	**Waluta**: dolar amerykański albo euro po dziennym kursie referencyjnym Europejskiego Banku Centralnego (pobieranym raz dziennie z ecb.europa.eu). **Dzienny limit** dla agenta płatnego za token (domyślnie **100**; od 10 do 500). **Żółty od** i **Czerwony od**: udział zużycia, przy którym wskaźnik kosztu lub bateria robi się żółta (domyślnie 75 %) i czerwona (90 %). **Powiadom przy czerwonym** (domyślnie włączone): powiadomienie na pulpicie, gdy agent robi się czerwony — raz dla każdego limitu aż do jego odnowienia albo raz dziennie dla kosztu.
Karty terminali nad mapą	Jedna karta na każdy terminal agenta w oknie.
Narożniki	Jak w oknach, zaokrąglone lub proste.
Uruchamiaj lokalne LLM razem z agentami	Strażnik w tle z sekcji Uruchamianie i zatrzymywanie z agentami.
W programie uruchamiającym	Wpis w programie uruchamiającym; wyłączenie go usuwa.
Agenci startują w	Folder, w którym startują agenci.
Funkcje AI: **Sprawdzaj nowe agenty**	Pytanie zainstalowanego agenta, gdzie nieznany agent trzyma swoje ustawienia (domyślnie włączone).
Funkcje AI: **Codzienne kontrole AI**	Raz dziennie agent szuka w sieci nowych wersji twoich serwerów MCP (domyślnie wyłączone; zużywa tokeny agenta). Zob. Smart Check.
Funkcje AI: **Agent do kontroli AI**	Claude Code lub Codex: wykonuje kontrole i wyszukiwania AI oraz przejmuje zadania **Zrób to z …** i **Napraw z …**.

Ustawienie	Co robi
Funkcje AI: **Lokalny pomocnik**	Agent z modelem twojego lokalnego LLM do pomocy AI, która nie potrzebuje sieci (wyłączony, dopóki go nie wybierzesz). Zob. Lokalny pomocnik.
Folder modeli	Jeden folder na wszystkie modele, z **Uporządkuj modele...**.
Dodatki MAPRISMA	**Szukaj dodatków online**, strona dodatków i **Konfiguracja**.
O programie	Wersja, autor i licencja; **Strona** otwiera projekt na GitHubie, a **Zgłoś problem** jego listę zgłoszeń.



Rysunek 11 · Ustawienia › O programie.

10.5 Czerwone limity w podglądzie Flutter

W podglądzie Flutter aplikacja raz na minutę sprawdza limity planu zainstalowanych agentów (5-godzinna sesja, tydzień...). Gdy któryś jest zużyty do czerwonego poziomu z Ustawienia › Koszty, mówi o tym powiadomienie na pulpicie, z zużytą częścią i czasem resetu limitu: raz na limit, aż się zresetuje, także po ponownym uruchomieniu. **Powiadom przy czerwonym** w Ustawieniach je wyłącza. Koszt agenta płacącego za tokeny nie jest jeszcze pokazywany w podglądzie, więc i jego powiadomienie.

10.6 Ostatnie zmiany i Cofnij po ponownym uruchomieniu

W podglądzie Flutter Ustawienia kończą się sekcją **Ostatnie zmiany**: ostatnie 20 zmian, które ta aplikacja zapisała w ustawieniach agentów (połączenia, serwery MCP, skille, ustawienia bezpieczeństwa, aktualizacje aplikacji), od najnowszej, z plikami, których dotyczyły. **Cofnij** przywraca to, czego od tamtej pory nie zmieniono ponownie — także po zamknięciu lub awarii aplikacji, gdy **Cofnij**

ostatniej zmiany już nie jest dostępne. Zmiana, której pliki od tamtej pory ponownie edytowano, nie ma **Cofnij**. Zmiana przerwana w połowie (aplikację zamknięto podczas zapisu) przywraca tylko pliki, które już zapisała. Kopie zapasowe zostają na miejscu; Cofnij tylko je czyta.

10.7 Pomoc

? (F1) otwiera pomoc: **Otwórz instrukcję** otwiera tę instrukcję (wydanie dołączone do twojej wersji, w twoim języku) w przeglądarce PDF; **Miejsce na dysku** (niżej); animacja o konfigurowaniu przez rysowanie; oraz LLM Monitor, towarzysząca wtyczka paska, która pokazuje, co robi lokalny LLM (mała animacja jego wpisu na pasku i panelu). Obie animacje są rysowane w kolorach twojego motywu i natychmiast podążają za zmianą motywu. Legenda ⓘ w nagłówku objaśnia każdą kropkę, linię i znacznik na mapie.

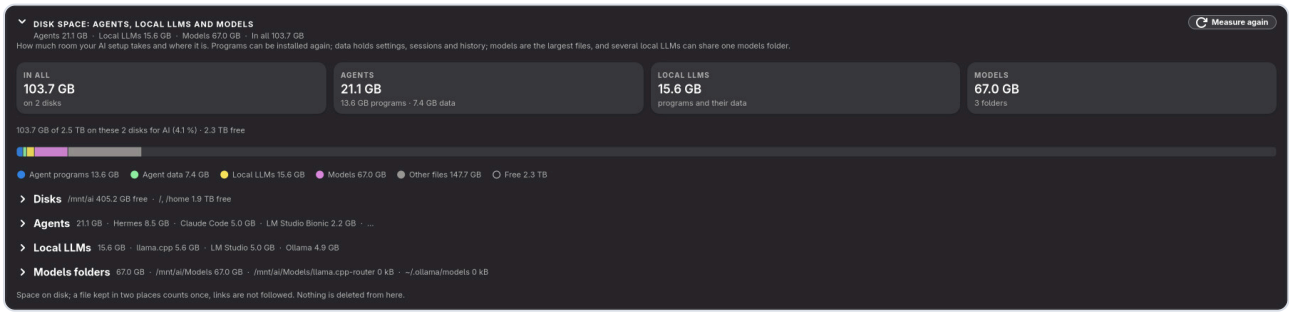
Wypełnione **i** w nagłówku otwiera **Informacje**: karty innych wtyczek autora, odczytywane z Omarchy Plugin Marketplace przy otwarciu strony, najwyżej raz na godzinę (bez sieci lista, którą zna ta wersja). Kliknięcie karty otwiera stronę wtyczki w marketplace.

10.8 Pomoc w podglądzie Flutter

W podglądzie Flutter **?** w nagłówku (lub F1) otwiera **Pomoc**: **Otwórz instrukcję** otwiera tę instrukcję w twoim języku (kopię dołączoną do aplikacji, inaczej angielską) w przeglądarce PDF pulpitu. Zakładki u góry Pomocy prowadzą do **Twój pierwszy agent**, **Co możesz z tym zrobić**, **Agenci** (lista agentów), **Dostawcy i modele** i **Połączenia**; **Użycie** pokazuje uruchomienia AI z ostatnich 30 dni. **Od czego zacząć** prowadzi do **Znajdź istniejącą AI**. **Mapa AI** w nagłówku wraca do mapy z każdej z tych stron.

10.9 Miejsce na dysku

Miejsce na dysku: agenci, lokalne LLM i modele pokazuje, ile miejsca zajmuje twoja konfiguracja AI i gdzie. Jest mierzone w tle przy każdym otwarciu pomocy (ponownie po dziesięciu minutach lub przez **Zmierz ponownie**) i tylko odczytywane — stąd nic nie jest usuwane. Nagłówek zawsze pokazuje sumy w jednym wierszu (*Agenci · Lokalne LLM · Modele · Razem*); kliknięcie zwija sekcję. **Dyski**, **Agenci**, **Lokalne LLM** i **Foldery modeli** zaczynają zwinięte, każdy ze swoją sumą i największymi elementami; kliknięcie je otwiera.



Rysunek 12 · Sumy, udział AI w całym miejscu na dysku i zwinięte sekcje z sumami.

- **Kafelki**: wszystko razem, potem programy agentów, dane agentów (ustawienia, sesje, historia), lokalne LLM (programy i ich własne dane) i modele.

- **Pasek**: całe miejsce dysków, na których jest konfiguracja AI. Kolorowe części to konfiguracja AI, szare to wszystko inne na tych dyskach, a pusta reszta jest wolna; wiersz nad paskiem podaje udział AI w procentach. Najedź na pasek, aby zobaczyć dokładne rozmiary.
- **Dyski**: każdy dysk z tym, ile zajmuje na nim konfiguracja AI, i ile jest wolnego.
- **Agenci**, **Lokalne LLM**, **Foldery modeli**: po jednym wierszu, największe najpierw. Otwórz wiersz, aby zobaczyć każdy folder z rozmiarem i liczbą plików i otworzyć go w menedżerze plików. Folder programu mówi, ile wersji jest przechowywanych; dla agenta z kilkoma starymi wersjami w mise wskazówka wspomina `mise prune`, które usuwa nieużywane. Folder modeli mówi, które lokalne LLM go używają, albo że zawiera tylko dowiązania do modeli w innym folderze.

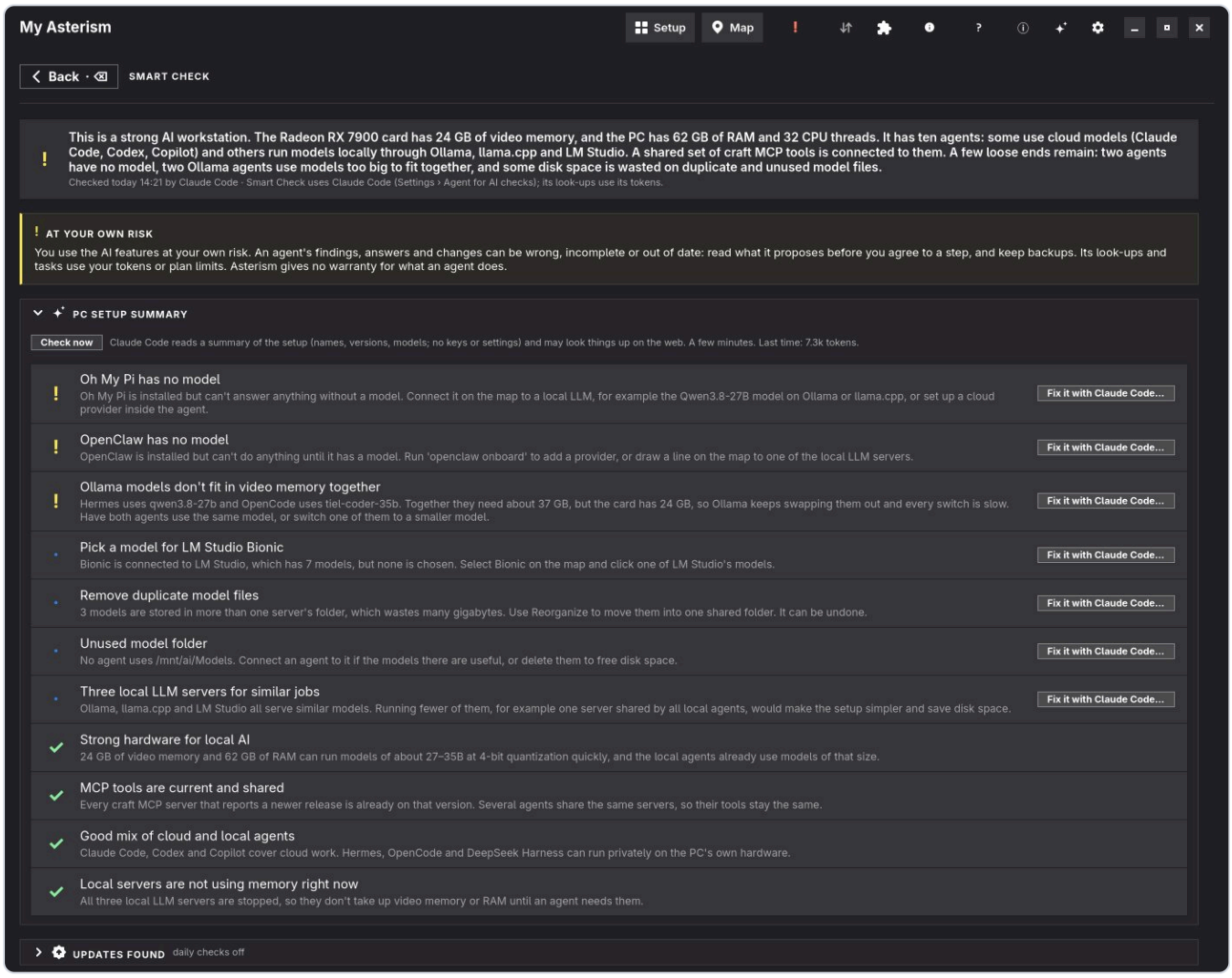
> **WSKAZÓWKA** Programy można zainstalować ponownie, a modele pobrać ponownie; dane agentów zawierają twoje sesje i ustawienia. Gdy brakuje miejsca, najpierw przejrzyj stare wersje programów i modele, których już nie używasz.

Rozmiary to miejsce na dysku: plik trzymany w dwóch miejscach liczy się raz, a dowiązania nie są śledzone.

ROZDZIAŁ 11

Smart Check

Smart Check pozwala zainstalowanemu agentowi online — Claude Code lub Codex — przyjrzeć się konfiguracji AI tego komputera, znaleźć aktualizacje twoich serwerów MCP i wykonać za ciebie naprawy, a także pokazuje, ile to kosztowało w tokenach. Lokalny pomocnik — agent z modelem twojego własnego lokalnego LLM — może przejąć pytania, które nie potrzebują sieci, na tym komputerze i bez tokenów. Każde uruchomienie pokazuje swój postęp w polu nad oknem i jest zachowywane w Protokole. Smart Check otwierają dwie iskierki w nagłówku; są widoczne tylko wtedy, gdy zainstalowany jest jeden z tych agentów. Ustawienia › **Agent do kontroli AI** wybiera, który.



Rysunek 13 · Smart Check po kontroli przez Claude Code: problemy (!), wskazówki (·) i to, co jest dobre (✓).

NA WŁASNE RYZYKO Ustalenia, odpowiedzi i zmiany agenta mogą być błędne, niepełne lub nieaktualne. Przeczytaj, co proponuje, zanim zgodzisz się na krok, i miej kopie zapasowe. Jego wyszukiwania i zadania zużywają twoje tokeny lub limity planu. Aplikacja nie daje żadnej gwarancji za to, co robi agent. Ta sama uwaga otwiera Ustawienia › **Funkcje AI**.

11.1 Podsumowanie konfiguracji PC

Sprawdź teraz daje agentowi krótki tekst o konfiguracji — karta graficzna i pamięć, agenci z ich modelami, lokalne LLM, serwery MCP z wersjami, skille i to, co znalazł Stan kolumn — i pozwala mu sprawdzać rzeczy w sieci. Nie ma w nim kluczy, nagłówków ani wartości ustawień. Trwa to minutę lub dłużej w tle; wiersz pod przyciskiem mówi, ile tokenów zużyła ostatnia kontrola. Odpowiedź to krótkie podsumowanie i lista wyników: **!** problemy, · wskazówki i ✓ to, co jest dobre. Odpowiedzi, które się nie potwierdzają, są pomijane.

Napraw z … obok problemu lub wskazówki otwiera agenta w karcie terminala w oknie z tym wynikiem jako zadaniem: sprawdza, czy tak jest, wyjaśnia i pyta cię przed każdym krokiem, który coś zmienia. Aplikacja uruchamia go tak, że musi pytać — Codex w piaskownicy tylko do odczytu, która prosi o więcej, Claude Code z każdym poleceniem i edycją ustawionymi na pytanie, bez względu na jego własną listę dozwolonych — i bez jego serwerów MCP. Wynik jest przekazywany jako dane do sprawdzenia, nie jako instrukcje. Przeniesienie serwerów MCP na nową wersję zostaje dla **Aktualizuj** aplikacji (niżej), które zmienia ustawienia wszystkich agentów naraz, z kopią zapasową i Cofnij.

11.2 Znalezione aktualizacje

Nowe wersje twoich serwerów MCP, z dwóch źródeł:

- **Codzienne kontrole AI** (Ustawienia › **Codzienne kontrole AI**, domyślnie wyłączone): raz dziennie agent szuka na oficjalnych stronach wydań nowszych wersji serwerów, których wersję aplikacja potrafi rozpoznać — aplikacji rozpakowanych w `~/Applications/<app>-<version>/` lub jako rozpakowany AppImage w dowolnym innym folderze (program w `<folder>/<app>-<version>/usr/bin/`) oraz pakietów npm lub PyPI przypiętych do wersji. **Sprawdź teraz** na tej karcie pyta od razu, także gdy codzienne kontrole są wyłączone. Każde znalezisko ma **Strona wydania** i **Zrób to z …**: agent pobiera nowy AppImage w karcie terminala, pytając przed każdym krokiem.
- **Własne skanowanie aplikacji**: nowsza wersja jest już na tym komputerze, ale agent wciąż uruchamia starą (to samo, co aktualizacja na karcie serwera w Konfiguracji).

Gdy nowa wersja jest na tym komputerze, jej wiersz to mówi i oferuje **Użyj…**: podgląd aktualizacji pokazuje, którzy agenci na nią przejdą; **Aktualizuj** zmienia ich wszystkich, zachowuje starą wersję w kopii zapasowej i potem testuje każdy serwer. Serwer, którego wszyscy agenci uruchamiają już nową wersję, znika z listy.

> **WSKAZÓWKA** Po działaniu rozpoczętym tutaj — aktualizacji lub zadaniu w karcie terminala — **Wstecz** przenosi cię z powrotem do Smart Check, z aktualnym stanem.

11.3 Gdy agent pracuje

Każde uruchomienie pomocy AI — Sprawdź teraz, Zapytaj agenta na stronach Modele i Dodaj serwer MCP, Sprawdź teraz dla aktualizacji, Zapytaj na karcie Wydajność lokalnego LLM — pokazuje pole na środku okna: kogo pytamy (agenta i jego model), jego kroki (animowany wskaźnik, który zmienia się w ✓, albo w ✕, gdy krok się nie udał), dotychczasowy czas i na końcu zużyte tokeny.

- **W tle** ukrywa pole; praca trwa dalej, a komunikat powie, kiedy się skończy.

- **Anuluj** natychmiast kończy agenta (jego program i wszystko, co uruchomił).
- Gdy coś się nie uda, pole zostaje z opisem błędu, dopóki go nie zamkniesz; tekst można zaznaczyć i skopiować.

Wyszukiwania w sieci (porady dotyczące modeli, aktualizacje MCP) działają z siecią i niczym więcej: bez serwerów MCP, konektorów, wtyczek i hooków agenta.

11.4 Lokalny pomocnik

Agent połączony z lokalnym LLM może odpowiadać na pomoc AI jednym z modeli tego LLM: na tym komputerze, bez sieci, bez tokenów planu i bez wysyłania czegokolwiek poza komputer. Odpowiada na to, co nie potrzebuje sieci: podsumowanie Smart Check, który serwer MCP pasuje oraz wskazówki na karcie Wydajność lokalnego LLM. Porady dotyczące modeli i aktualizacje serwerów MCP nadal trafiają do agenta online, bo potrzebują sieci.

Aplikacja proponuje jeden — największy model czatu, jaki zmieści twoja karta graficzna (modele mowy, embeddingów i tokenizerów są pomijane) — w Smart Check, z **Użyj**. Ustawienia › **Funkcje AI** › **Lokalny pomocnik** wymienia każdego agenta i model, który może nim być, proponowany jako pierwszy, oraz **Wyłączone**. Nic nie jest używane, dopóki tego nie wybierzesz. Zanim odpowie, aplikacja uruchamia lokalny LLM i ładuje model (krok w polu postępu). Pomocnik działa bez narzędzi, bez serwerów MCP, bez rozszerzeń i bez plików projektu. Pomocnikiem może być każdy agent, którego dodatek mówi, jak zadać mu jedno pytanie offline; Pi jest pierwszy.

11.5 Kontrole AI i lokalny pomocnik w podglądzie Flutter

W podglądzie Flutter w Ustawieniach › **Funkcje AI** są **Codzienne kontrole AI** (domyślnie wyłączone), a pod nimi wynik ostatniej kontroli z przyciskiem **Sprawdź teraz**, który pyta od razu, także gdy codzienne kontrole są wyłączone. Kontrola działa w tle po otwarciu okna i jest sprawdzana co godzinę; agent do kontroli AI jest pytany najwyżej raz dziennie (po nieudanym wyszukiwaniu ponownie po sześciu godzinach). To, co znajdzie, pojawia się w stanie kolumny MCP: **Strona wydania** otwiera wydanie w przeglądarce, **Zrób to z …** najpierw pokazuje zadanie w takiej postaci, w jakiej zostanie wysłane, i uruchamia je w karcie terminala dopiero po **Wznów**, a **Użyj…** otwiera to, co znaleziono na tym komputerze, gdy nowsza wersja już tu jest. Sprawdzane są tylko serwery, których wersję aplikacja potrafi rozpoznać (zob. Znalezione aktualizacje); inne serwery nie są wysyłane.

Lokalnego pomocnika wybiera się w Ustawieniach › **Funkcje AI** › **Lokalny pomocnik**; propozycja dla Twojej karty graficznej jest oznaczona jako **(sugerowany)**. Po wybraniu jest proponowany jako pierwszy w **Pytaj** na karcie Wydajność lokalnego LLM.

11.6 Protokół

Każde uruchomienie pomocy AI jest zachowywane, żebyś ty — albo ktoś, kto dobrze zna konfigurację — mógł przejrzeć, co zostało zrobione. Każdy wpis pokazuje, kiedy działał, agenta i model, wynik (udało się, nie udało się, anulowano), czas i tokeny. Po otwarciu pokazuje cały tekst: opcje, które zamknęły agenta w granicach, kroki, każde pytanie dokładnie tak, jak zostało wysłane, z odpowiedzią agenta, co

aplikacja zmieniła na podstawie odpowiedzi (z kopią zapasową, do ręcznego Cofnij) oraz, dla zadania w karcie terminala, co terminal pokazywał, gdy się skończyło. **Kopiuj** umieszcza wpis w schowku do zgłoszenia lub drugiej opinii; **Otwórz folder protokołu** pokazuje pliki.

Protokół przechowuje ostatnie 200 uruchomień w `~/.local/state/ai-agents-map/ai-help/`, czytelnych tylko dla ciebie. Pytania zawierają nazwy, liczby i ustawienia, nigdy klucze; tekst terminala zawiera to, co było na ekranie.

11.7 Statystyki tokenów

Tokeny zużyte dziś i w ostatnich 30 dniach, wykres dla każdego dnia oraz sumy według agenta i według wtyczki (inne wtyczki Omarchy, które używają tego samego pomocnika AI, też się tu pojawiają). **Wartość API** to koszt tych uruchomień po cenach API, a wiersz pod nią mówi, czy pokrywa to twoja subskrypcja, czy obciąża to klucz API. Zadanie w karcie terminala liczy się we własnej sesji swojego agenta; te tokeny pokazuje strona szczegółów agenta.

Poniżej jedna karta na każdy rodzaj zadania — kontrolę konfiguracji, aktualizacje serwerów MCP, wyszukiwanie nowych agentów, naprawy — z jej uruchomieniami, tokenami i najnowszymi wynikami.

ROZDZIAŁ 12

Karty grup i dodatki na mapie

12.1 Rozwijanie kategorii

W podglądzie Flutter każda kolumna mapy AI ma kartę grupy. Przerywana ramka oznacza brak konfiguracji; liczba pochodzi z lokalnego wykrywania. Zastępcze karty są wyłącznie wizualne, nigdy połączonymi węzłami ani danymi dla AI. Użyj Rozwiń, dwukliku lub Enter na aktywnej karcie. Grupa pojawia się sama na środku. Wstecz, Esc, Backspace lub tło przywraca powiększenie i wybór.

12.2 Zarządzanie dodatkami

Grupy pokazują Włączone, Wyłączone, Znalezione na tym komputerze i Niezainstalowane zależnie od zawartości. Menu lub prawy przycisk oferuje przełączanie, szczegóły, nowszą podpisaną wersję lub logowanie do dostawcy przez zainstalowany agent. Szczegóły opisują odczyt, wysyłanie i możliwości. Zmiany działają od razu z Cofnij i są blokowane podczas własnych sesji agentów tej aplikacji lub aktywnej instalacji. Agenty w innych terminalach nie blokują zarządzania dodatkami. Szukaj dodatków online jawnie ładuje podpisany katalog; samo rozwinięcie tego nie robi. Sprawdź każdą instalację i aktualizację. Nowe dodatki pozostają wyłączone. Usunięcie dodatku użytkownika tworzy kopię i zachowuje programy oraz dane; cofnąć można do zamknięcia aplikacji. Dołączone dodatki należą do pakietu.

Po potwierdzonym usunięciu kopie starsze niż 30 dni mogą być czyszczone. Kopie z aktywnym Cofnij pozostają; łącza, obce foldery i foldery innych właścicieli są pomijane. Cofnij działa do zamknięcia aplikacji. Przy pierwszym użyciu Agenty w powitaniu otwierają tę samą grupę ze stałym paskiem Włącz / Później; nie kończą powitania ani nie otwierają zwykłego widoku.

Zainstaluj z pliku... bierze dodatek .zip pobrany przez ciebie. Decyduje jego własny podpis, tak jak przy pobraniu z katalogu: plik jest najpierw sprawdzany i pokazywany jak pozycja katalogu (nazwa, wersja, rodzaj, co czyta i wysyła oraz czy zastępuje zainstalowaną wersję), a nic nie jest instalowane, dopóki nie potwierdzisz. Wtedy plik jest czytany ponownie i musi to być wciąż ten sprawdzony. Na razie w podglądzie Flutter można używać tylko dodatków agentów i dostawców.

12.3 Pierwsze użycie i istniejące funkcje

Powitanie nad mapą zawiera Sprawdź ten komputer / Pomiń. Zamontuj istniejące dyski przed wyszukiwaniem. Agenty otwierają się z zaznaczonymi wykrytymi pozycjami i stałym paskiem Włącz / Później na dole. Włącz jest jedną operacją zbiorczą z jednym Cofnij; Później zachowuje wybory. Inne włączone typy dodatków pozostają. Kolejne uruchomienia nie rozwijają grupy automatycznie. Modele prowadzą do pobierania/importowania i ponownego użycia folderów; MCP i umiejętności do Połączeń. Nowe definicje MCP i foldery umiejętności nadal czekają na migrację. Rakieta otwiera przewodnik pierwszego agenta. Grupy zastępują strony Konfiguracja i Dodatki.

ROZDZIAŁ 13

Edycja mapy SI we Flutterze

Przeciągnij agenta do dostawcy, serwera MCP lub zestawu umiejętności albo rysuj w drugą stronę. Wybierz agenta, a następnie model połączonego dostawcy. Linia od dostawcy do jego modelu otwiera wybór agentów; obsługiwany dostawca lokalny może też ładować bez agenta. Każda zmiana przechodzi przez Podgląd, Zastosuj, postęp i wynik z Cofnij. Anulowanie niczego nie zapisuje.

Wybierz pole, aby pokazać pomarańczowe uchwyty. Przeciągnij uchwyt do zgodnego dostawcy lub modelu albo na puste miejsce, aby rozłączyć. Uchwyt modelu dostawcy przełącza wszystkich skonfigurowanych użytkowników albo rozłącza ich i zwalnia model, jeśli to obsługiwane. Zajęty silnik odmawia zwolnienia. Dostawcy online używają zarejestrowanego polecenia logowania lub wylogowania w terminalu po przeglądzie.

Najechanie podświetla sąsiadów. Shift-klik dodaje pola, najwyżej jednego agenta. Przypnij utrzymuje podświetlenie. Zwiń kartę strzałką, przeciągnij nagłówek lub użyj menu. Przeciągnij uchwyt kolejności albo wybierz Przesuń w górę / Przesuń w dół. Układ i przypięcia zapisują się oddzielnie od połączeń; karty zadań zachowują wybór i powiększenie.

Klawiaturą otwórz Działania, wybierz Połącz lub Przenieś połączenie i aktywuj cel. Rozłącz usuwa oczekujące połączenie; Anuluj lub Esc porzuca zmianę. Strzałki, menu przesuwania, Przypnij i opcje kolejności zastępują przeciąganie. **Szczegóły** w menu pola (lub symbol szczegółów) pokazuje zarządzanie dla każdego pola. Pola agentów pokazują odznakę limitu lub kosztu oraz pole statystyk użycia w tytule, a także zwarte symbole dla podagentów (z liczbą, gdy istnieją), ruchu sieciowego (z bieżącą prędkością przychodzącą i wychodzącą w podpowiedzi, gdy są połączenia; kliknięcie otwiera okno ruchu z sumami, miejscami docelowymi i programami), użycia, zatrzymania (czerwony, widoczny tylko gdy agent działa; zatrzymuje instancję w terminalu aplikacji albo zewnętrzną instancję) i szczegółów.

Ustawienia › **Mapa**: Wszystkie światła, Światła agentów i Światła wyłączone sterują szacowaną aktywnością. Gasną bez aktywnych linii, przy ukrytej mapie, utracie fokusu lub ograniczeniu ruchu. Błędne linie pokazują ograniczone wskazówki z Napraw lub Rozłącz. Cofnij przywraca konfigurację i metadane błędów; nie cofa ładowania, zwalniania ani logowania. Częściowa operacja zachowuje kopię i Cofnij.

ROZDZIAŁ 14

Karty zadań na mapie

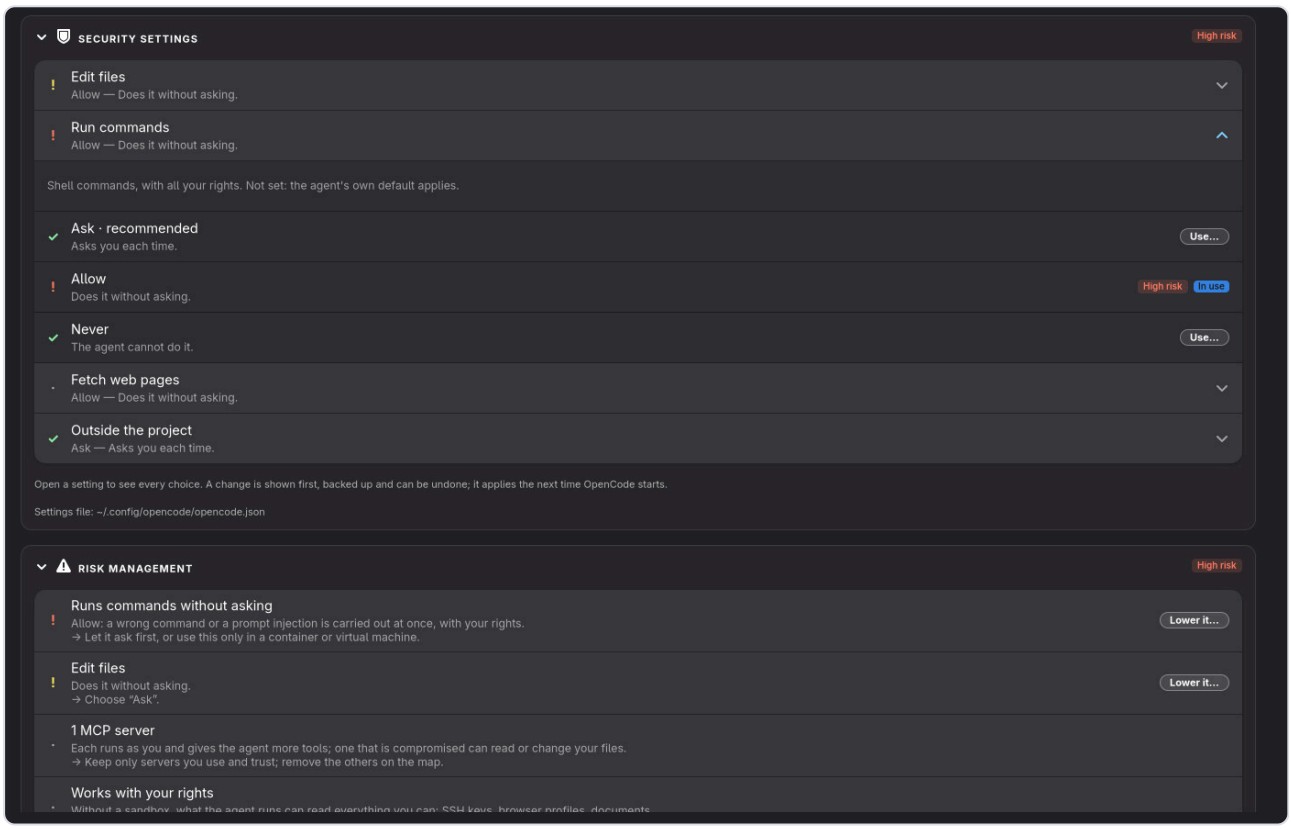
Połączenia, działania agentów, usuwanie modeli i zmiany dodatków otwierają kartę zadania w podglądzie Flutter. Gdy obszar mapy ma co najmniej 900 pikseli logicznych, a elementy zadania są znane, po lewej pojawia się mapa tylko do odczytu. Pokazuje te elementy i ich bezpośrednich sąsiadów z liniami do karty. Puste kolumny są pomijane. Mapa główna zachowuje powiększenie, pozycję i wybór. Przegląd dostawców zajmuje pełną szerokość; działania dla pojedynczego silnika pokazują pół karty, gdy ich elementy mapy są znane.

Sprawdź zmiany przed zatwierdzeniem. Ta sama karta pokazuje postęp i wynik; użyj Cofnij, jeśli jest dostępne. Wstecz lub Esc odrzuca podgląd albo zamyka zakończoną kartę. Nie zamykają trwających zadań; tylko operacje obsługujące anulowanie pokazują Anuluj. Wyszukiwanie i pobieranie modeli, katalog i przewodnik pierwszego agenta zajmują pełną szerokość. Dotyczy to też wąskich okien i zadań bez znanych elementów. Ustawienia, terminale, użycie i szczegóły podagentów zachowują dotychczasowe widoki.

ROZDZIAŁ 15

Kontrola stanu i bezpieczeństwa

Agent programistyczny czyta i zapisuje pliki oraz uruchamia polecenia na twoim komputerze. Ile może zrobić *bez pytania cię*, zapisano w jego własnych ustawieniach, innymi słowami dla każdego agenta. **Kontrola stanu i bezpieczeństwa** zbiera to razem: na stronie szczegółów każdego agenta i każdego lokalnego LLM cztery karty pokazują, czy działa, co może robić, co to znaczy prostymi słowami i co zmniejsza ryzyko.



Rysunek 14 · Ustawienia bezpieczeństwa OpenCode z otwartym **Uruchamianie poleceń** i początek jego zarządzania ryzykiem.

15.1 Czytanie kart

Plakietki na zwiniętych kartach to podsumowują: **W porządku**, **Do sprawdzenia** lub **Nieskonfigurowany** przy Stanie oraz **Niskie ryzyko**, **Średnie ryzyko** lub **Wysokie ryzyko** przy Ustawieniach bezpieczeństwa i Zarządzaniu ryzykiem. Otwórz kartę, aby zobaczyć szczegóły:

Karta	Co pokazuje
Stan	Każde sprawdzenie konfiguracji z ✓ albo co zrobić dalej: zainstalowany, zalogowany, jego model i lokalny LLM, serwery MCP i skille.
Ustawienia bezpieczeństwa	Ustawienia agenta, które decydują, co może robić samodzielnie, każde z obecną wartością i zdaniem o tym, co ona znaczy. Znak przed nim podaje ryzyko: ✓ bezpieczne, · niskie, żółty **!** średnie, czerwony **!** wysokie.

Karta	Co pokazuje
Zarządzanie ryzykiem	Co ustawienia znaczą razem, najpoważniejsze najpierw, każde z powodem i → tym, co je zmniejsza. Istniejące zabezpieczenia są wymienione z ✓.
Co znaczą te słowa	Sandbox, zatwierdzanie, reguły zezwalania i zakazu oraz inne słowa z kart, objaśnione (zob. Słowa objaśnione).

Reguły zezwalania i zakazu oraz zaufane foldery agenta (na przykład lista `permissions.allow` w Claude Code) są wymienione pod ustawieniami, jedna reguła w wierszu.

15.2 Zmiana ustawienia

1. Kliknij ustawienie, aby je otworzyć. Widzisz, czym steruje, i każdy wybór z jego ryzykiem; używany jest oznaczony jako **W użyciu**, bezpieczniejszy domyślny jako **zalecany**.
2. Kliknij **Użyj …** obok wyboru lub **Zmniejsz …** obok ryzyka. Strona pokazuje zmianę słowami (stare → nowe), ryzyko przed i po oraz **Co się zmienia**: wiersze pliku ustawień agenta.
3. Kliknij **Zmień**. Plik jest kopiowany zapasowo i zapisywany; powiadomienie oferuje **Cofnij** przez 30 sekund. Agent używa nowego ustawienia przy następnym starcie.

> **UWAGA** Zmienia się tylko ten jeden klucz; komentarze i wszystko inne w pliku zostają bez zmian. Ustawienie, które wymaga więcej niż jednego wiersza w pliku — jak Hermes uruchamiający polecenia w Dockerze — jest pokazane i objaśnione, z miejscem, gdzie je zmienić, ale nie jest zmieniane stąd.

15.3 Bezpieczeństwo w podglądzie Flutter

W podglądzie Flutter szczegóły zainstalowanego agenta mają **Ustawienia bezpieczeństwa**. Karta pokazuje odznakę ryzyka, każde ustawienie z jego wyborami na liście (kolorowa kropka podaje ryzyko każdego wyboru, bezpieczniejsza wartość domyślna jest oznaczona jako **zalecany**; ustawienie, którego plik nie wymienia, mówi, że obowiązuje własna wartość domyślna agenta), reguły zezwalania i zabraniania, plik ustawień oraz **Zarządzanie ryzykiem** z **Zmniejsz…** tam, gdzie jedno ustawienie zmniejsza ryzyko. Wybór innej wartości lub **Zmniejsz…** otwiera **Podgląd**, którego używa zmiana połączeń: wiersze pliku ustawień, które się zmieniają. **Zastosuj** robi kopię pliku i go zapisuje, potem z **Cofnij**; zmienia się tylko ten jeden klucz, komentarze i reszta pliku zostają bez zmian.

15.4 Stan w podglądzie Flutter

W podglądzie Flutter szczegóły zainstalowanego agenta i każdy lokalny LLM w **Dostawcy i modele** mają kartę **Stan**: **W porządku**, **Do sprawdzenia** lub **Nieskonfigurowany**, jedno zdanie i każdą kontrolę — zainstalowany, zalogowany (sprawdza się tylko, czy jest plik dostępu albo klucz), model, jego lokalne LLM, nieudane połączenia, serwery MCP i skille; przy lokalnym LLM, czy działa, jego modele i połączeni agenci. Kontrola, która nie jest w porządku, mówi, co zrobić dalej, z poleceniem do **Kopiuj**, gdy takie jest. Karta otwiera się sama, gdy jest coś do zrobienia.

15.5 Co jest sprawdzane u każdego agenta

Agent	Pokazywane i zmienialne ustawienia	Plik
Claude Code	Tryb uprawnień (Najpierw pytaj, Tylko planuj, Tylko dozwolone, Edytuj bez pytania, Automatyczna ocena, Pomiń wszystkie kontrole), sandbox dla poleceń, blokada trybu obejścia; reguły zezwalaj/pytaj/zakaż	`~/.claude/settings.json`
Codex	Kiedy Codex pyta, sandbox (tylko odczyt, folder projektu, brak), sieć w sandboxie; zaufane foldery	`~/.codex/config.toml`
GitHub Copilot	Tryb uprawnień (Najpierw pytaj, Automatyczna ocena, Zezwalaj na wszystko); zaufane foldery, strony bez pytania	`~/.copilot/settings.json`
Cursor CLI	Zatwierdzanie, sandbox, sieć w sandboxie, wyszukiwanie w sieci bez pytania; reguły zezwalaj/zakaż	`~/.config/cursor/cli-config.json`
Hermes	Niebezpieczne polecenia (zawsze pytaj, automatyczna ocena, nigdy nie pytaj), ukrywanie sekretów; gdzie działają polecenia (tylko pokazywane)	`~/.hermes/config.yaml`
OpenCode	Pytaj, zezwalaj lub nigdy dla edycji plików, uruchamiania poleceń, pobierania stron i pracy poza projektem	`~/.config/opencode/opencode.json`
Pi	Jego narzędzia: czytanie, pisanie i uruchamianie poleceń albo tylko czytanie. Pi z założenia nigdy nie pyta, więc jego narzędzia są jego uprawnieniami.	`~/.pi/agent/settings.json`

Dla innych agentów karta pokazuje ich stan i serwery MCP oraz mówi, gdzie zajrzeć we własnych ustawieniach agenta. Dla **lokalnego LLM** pokazuje, kto może się z nim połączyć: tylko ten komputer (`127.0.0.1` , tak jak powinno być) czy każde urządzenie w twojej sieci.

15.6 Panowanie nad ryzykiem

Poziom ryzyka to najpoważniejszy punkt, jaki znajduje karta:

- **Wysokie ryzyko** — może robić poważne rzeczy bez pytania: uruchamiać dowolne polecenie albo pracować bez sandboxa i bez pytań. W porządku w jednorazowym kontenerze lub maszynie wirtualnej; na własnym komputerze niech najpierw pyta.
- **Średnie ryzyko** — niektóre rzeczy dzieją się bez pytania, na przykład edycja plików w projekcie, albo automatyczna ocena decyduje za ciebie.
- **Niskie ryzyko** — pyta przed wszystkim, co ma znaczenie, albo w ogóle tego nie może.

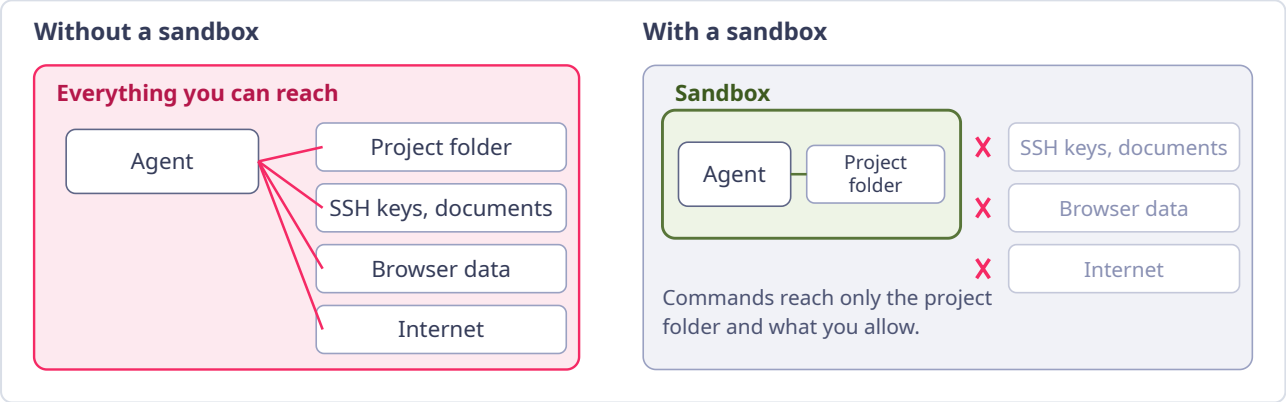
Dobre nawyki, niezależnie od ustawień:

- **Najpierw pytaj przy poleceniach** na własnym komputerze; pozwalaj na pojedyncze, nieszkodliwe polecenia (jak `git status`) regułą zezwalania zamiast na całą powłokę.

- **Włącz sandbox** tam, gdzie agent go ma (Claude Code, Codex, Cursor CLI), i zostaw w nim sieć wyłączoną, dopóki jakieś zadanie jej nie potrzebuje.
- **Uruchamiaj agentów w folderze projektu**, nie w folderze domowym, i trzymaj klucze i hasła z dala.
- **Mało serwerów MCP**: tylko te, których używasz i którym ufasz. Każdy działa z twoimi uprawnieniami.
- **Pracę bez nadzoru** (tryb obejścia, „YOLO", Pi ze wszystkimi narzędziami) uruchamiaj w kontenerze lub maszynie wirtualnej.
- **Lokalne LLM na** `127.0.0.1`: ich serwery nie mają hasła.

> **OSTROŻNIE** Automatyczna ocena (tryb *auto* w Claude Code, *assisted* w Copilot, *auto-review* w Cursor, *smart* w Hermes) to druga AI, która ocenia każdy krok. Oszczędza pytań, ale to ocena, nie mur.

15.7 Słowa objaśnione



Rysunek 15 · Co zmienia sandbox: polecenia sięgają tylko do folderu projektu i tego, na co pozwolisz.

Sandbox	Odgrodzona przestrzeń, w której działają polecenia agenta. W niej polecenie może dotknąć tylko folderu projektu (i tego, na co pozwolisz) — nie reszty folderu domowego, twoich kluczy SSH ani danych przeglądarki, a często też nie internetu. Bez sandboxa polecenie agenta może wszystko, co możesz ty.
Zatwierdzanie (najpierw pytaj)	Zanim agent zmieni plik lub uruchomi polecenie, pokazuje, co chce zrobić, i czeka na twoje tak lub nie. Im mniej może zrobić bez pytania, tym więcej kontroli zachowujesz — i tym częściej pyta.
Reguły zezwalania i zakazu	Listy poleceń lub narzędzi, których agent może używać bez pytania (zezwalaj) lub nigdy (zakaż). Zakaz wygrywa z zezwoleniem. Reguła zezwalająca na dowolne polecenie powłoki wyłącza zatwierdzanie dla powłoki.
Zaufany folder	Folder, w którym według ciebie agent może pracować. Tam używa też własnych instrukcji i ustawień folderu, które mogą pochodzić od kogoś innego, gdy klonujesz projekt.
Dostęp do sieci	Czy polecenia mogą sięgać do internetu. Kod, który czyta twoje pliki i może wysyłać dane, mógłby je wynieść; instalowanie pakietów też wymaga sieci.
Obejście („YOLO”)	Wszystko bez pytania i bez granic. Tylko dla jednorazowej VM lub kontenera.
Automatyczna ocena	Drugi model AI przepuszcza nieszkodliwe działania i pyta o resztę.
Prompt injection	Instrukcje ukryte na stronie, w pliku lub odpowiedzi narzędzia, które próbują skłonić agenta do czegoś, o co nie prosiłeś. Pytanie najpierw, sandbox i brak sieci je zatrzymują.
Ukrywanie sekretów	Tekst wyglądający jak hasło, token lub klucz jest maskowany, zanim model go zobaczy lub trafi on do logu.
Narzędzia	Co agent w ogóle może robić: czytać, pisać, uruchamiać polecenia, szukać w sieci. Narzędzia, którego nie ma, nie da się nadużyć.
Serwer MCP	Mały program, który daje agentowi dodatkowe narzędzia. Działa jako ty, z twoimi uprawnieniami.
127.0.0.1 (localhost)	Adres, do którego sięgają tylko programy na tym komputerze. Serwer na `0.0.0.0` jest osiągalny z każdego urządzenia w twojej sieci.

ROZDZIAŁ 16

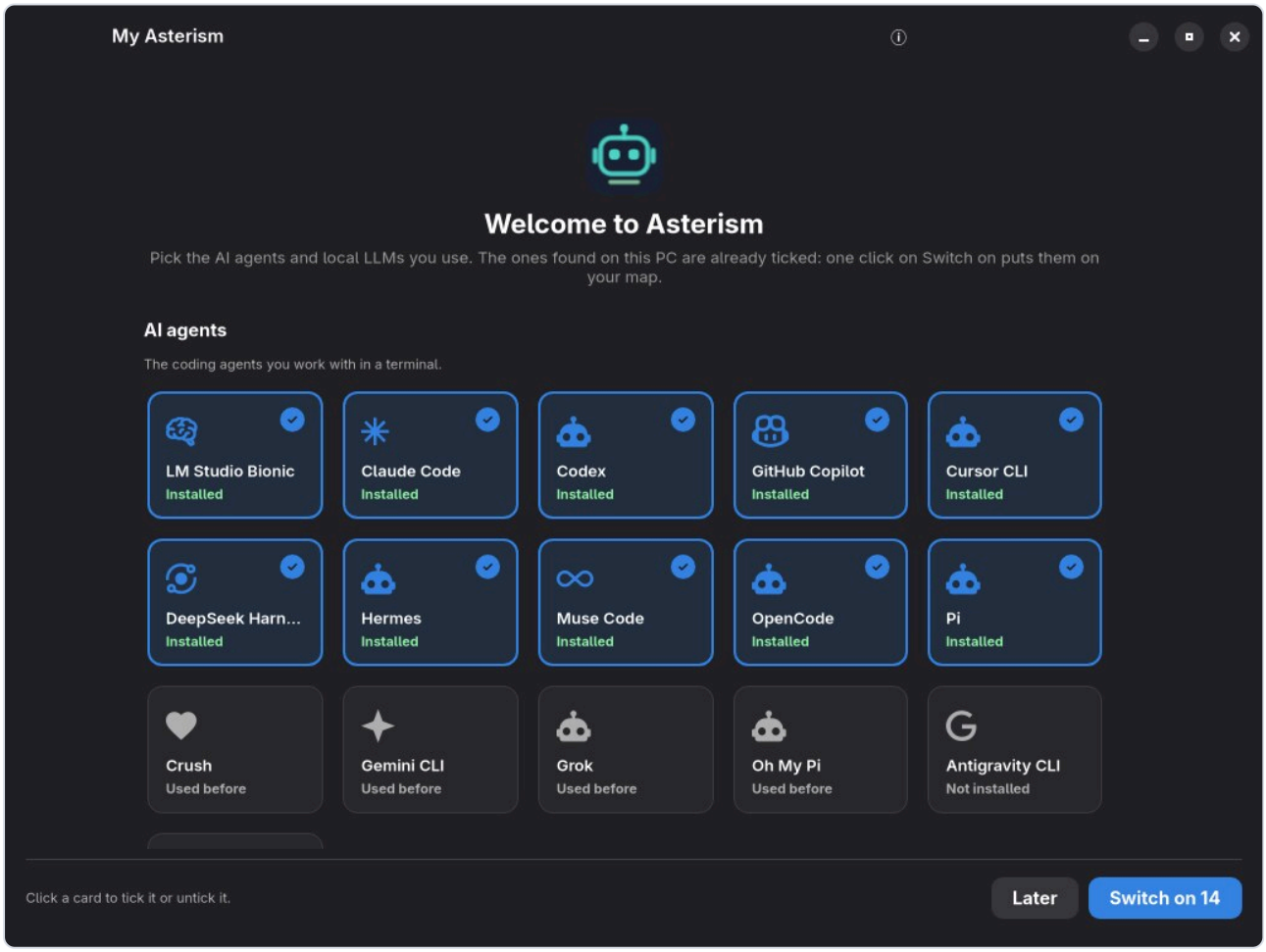
Dodatki

Wszystko, co Maprisma rozumie o agencie lub dostawcy LLM, to **adapter**: jak go znaleźć i zainstalować, jego sesje, ustawienia bezpieczeństwa, serwery MCP i skille. W Maprisma każdy adapter jest **dodatkiem**: pakiety przynoszą wszystkie, podpisane i wyłączone, a ty włączasz te, których używasz. Dodatki przynoszą też nowych agentów, nowych dostawców i funkcje wykraczające poza liczby, jak **Insights** (tematy sesji i podsumowanie od Claude). Wtyczka AI Agents Map ma adaptery wbudowane i nie ma dodatków.

16.1 Konfiguracja Maprisma

Nowy Maprisma otwiera się na swojej **stronie powitalnej**. Każdy agent i lokalny LLM, którego Maprisma zna, to karta z własną ikoną, w trzech grupach: **Agenci AI**, **Lokalne LLM** i **Dostawcy online** (nazwy Anthropic, OpenAI i pozostałych na mapie). Każda karta mówi, czy program tu jest:

- **Zainstalowany** (zielony) — znaleziony tak, jak znajduje go mapa; karta jest już zaznaczona;
- **Używany wcześniej** — teraz niezainstalowany, ale jego ustawienia wciąż są;
- **Niezainstalowany** — zaznacz go, aby potem zainstalować go z jego karty.



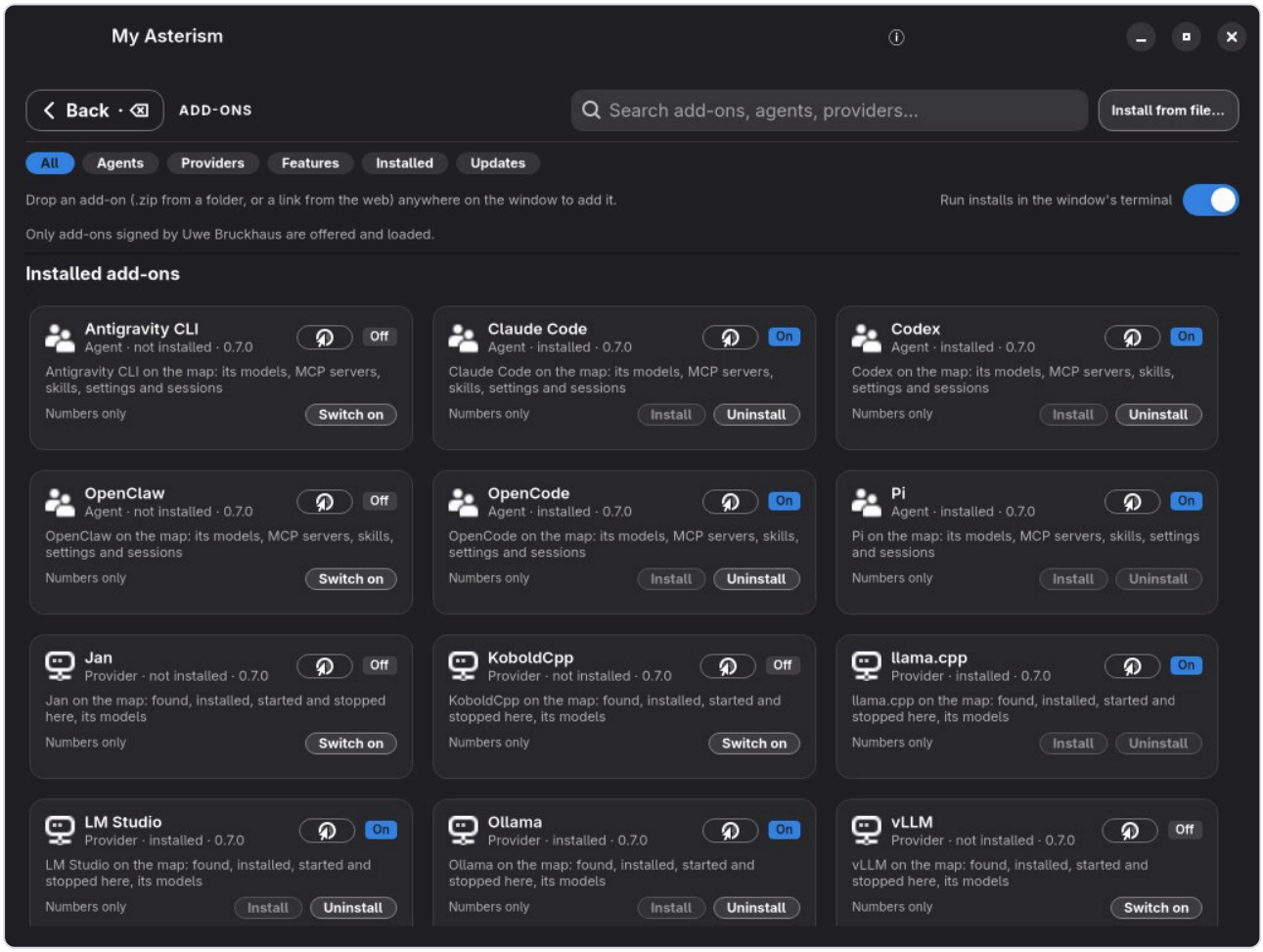
Rysunek 16 · Strona powitalna: znalezione programy są na początku i są zaznaczone.

Kliknij kartę (lub naciśnij na niej `Spację`), aby ją zaznaczyć lub odznaczyć; zaznaczona karta ma niebieską ramkę i haczyk. **Włącz** (z liczbą zaznaczonych kart) ładuje je i pokazuje mapę; jeśli zaznaczyłeś programy, które nie są zainstalowane, otwiera się **Konfiguracja**: ··· na ich kartach › **Zainstaluj…**. **Później** zaczyna z pustą mapą. Dopóki nic nie jest włączone, Maprisma przy każdym starcie otwiera się ponownie na stronie powitalnej. Potem te same karty są w **Konfiguracji** (przycisk w nagłówku), gdzie agentów i dostawców włącza się i wyłącza w dowolnym momencie.

16.2 Strona dodatków

Otwórz ją przyciskiem z puzzlem w nagłówku, przez Ustawienia › **Dodatki** lub z dowolnej wskazówki „dostępny dodatek”. Pokazuje, w zwijanych kartach grup, które zaczynają otwarte:

- **Proponowane dla tego komputera** — dodatek, którego agent lub dostawca jest tu zainstalowany, ale jeszcze nie jest pokazywany;
- **Zainstalowane dodatki** — włączony, wyłączony lub *niezaładowany* z powodem (na przykład zmieniony plik); agenci i dostawcy oddzieleni linią;
- **Dostępne** — pozostałe dodatki z katalogu;
- **Wbudowane w Maprisma** — agenci i dostawcy, których nie przynosi żaden zainstalowany dodatek (gdy uruchamiasz Maprisma z folderu źródłowego bez dodatków).



Rysunek 17 · Zainstalowane dodatki: włączone lub wyłączone, z Zainstaluj i Odinstaluj dla programu.

Karta włączonego agenta lub lokalnego LLM ma **Zainstaluj** i **Odinstaluj** dla samego programu (to, co nie ma zastosowania, jest wyszarzone), a każda karta ze stroną domową ma przycisk z globusem, który otwiera ją w przeglądarce.

Wyszukiwanie znajduje nazwy, opisy i to, co dodatek czyta; filtry powyżej zawężają listę do agentów, dostawców, funkcji, zainstalowanych lub tych z aktualizacją. Kliknij kartę, aby zobaczyć szczegóły: co czyta, co wysyła, co może robić i jego poziom prywatności.

16.3 Instalowanie, włączanie i wyłączanie, aktualizowanie, usuwanie

Zainstaluj (lub **Pobierz dodatek**) najpierw pokazuje, co dodatek czyta, wysyła i może robić; dopiero po twoim OK jest pobierany, sprawdzany i włączany. **Wyłącz** i **Włącz** działają od razu, bez restartu: jego agenci, dostawcy, sekcje i karty pojawiają się lub znikają wszędzie. **Aktualizuj** zastępuje go nowszą wersją z katalogu po sprawdzeniu nowej; **Usuń** przenosi go do kosza. **Zainstaluj z pliku…** przyjmuje zip z dodatkiem pobrany przez ciebie; obowiązują te same kontrole.

Gdy katalog ma nowsze wersje zainstalowanych dodatków, karty w **Konfiguracji** pokazują *Aktualizacja dodatku*, stan kolumny wymienia **Aktualizacje dodatków gotowe** z przyciskami **Zaktualizuj wszystko** i **Dodatki**, a strona dodatków pokazuje pasek z **Zaktualizuj wszystko**. Menu ⋯ karty (lub kliknięcie jej prawym przyciskiem) ma też **Zaktualizuj dodatek do** *wersja*, dla tego jednego dodatku. Zaktualizuj wszystko aktualizuje je po kolei, z tymi samymi kontrolami, i jednym komunikatem mówi, które zaktualizowano. Naraz działa tylko jedna instalacja lub aktualizacja; inna uruchomiona w tym czasie prosi o ponowną próbę za chwilę.

> **TYLKO OD AUTORA.** Maprisma ładuje tylko dodatki podpisane przez Uwe Bruckhausa (odcisk klucza `SHA256:e5cC634Ke3Our5LAeSc88qs22NKZEHTncBW5Ig3amWQ`). Zanim cokolwiek z dodatku zostanie uruchomione, podpis jest sprawdzany narzędziem `ssh-keygen` z OpenSSH, każdy plik jest porównywany z jego sumą SHA-256 i żaden plik nie może zostać dodany. Pobrany plik musi ponadto pasować do sumy SHA-256 w podpisanym katalogu.

16.4 Poziomy prywatności

Poziom	Co dodatek może czytać lub wysyłać
Tylko liczby	Liczniki, czasy, tokeny, zmienione wiersze, koszt — to, co Maprisma pokazuje bez dodatków.
Tytuły sesji	Tytuły, które agent sam nadał swoim sesjom (nigdy rozmowy).
Tytuły i podsumowanie od Claude	Te tytuły i liczby, wysłane do twojego własnego Claude Code — tylko po kliknięciu, po podglądzie tego, co dokładnie zostanie wysłane.

Dodatek nigdy nie może zrobić więcej, niż pozwala jego poziom. Dodanie agenta lub dostawcy albo instalacja programu to osobne uprawnienia, które wymienia jego karta.

16.5 Rodzaje dodatków

Rodzaj	Co przynosi
Agent	Agenta programistycznego: gdzie trzyma ustawienia, sesje, serwery MCP i skille, jak się go instaluje i jak się loguje. Wszystko inne Maprisma buduje z tego — kartę w Konfiguracji, jego pole na mapie, Użycie i tokeny, Projekty, Stan i Bezpieczeństwo.
Dostawca	Lokalny program LLM lub dostawców online: jak go znaleźć i zainstalować i jak agenci się z nim łączą. Maprisma buduje jego kartę, pole na mapie i stronę szczegółów — w tym **Użycie i tokeny**: co twoi agenci przez niego wysłali, z ich własnych liczb sesji.
Rozszerzenie	Więcej informacji o agencie lub dostawcy, którego przynosi inny dodatek, bez zastępowania go — na przykład jego plan i limity albo karta na jego stronie szczegółów. Wypełnia tylko to, co dodatek agenta zostawia puste; strona dodatku mówi, co dodaje i co z tego nie jest używane.
Funkcja	Coś obejmującego wielu agentów, jak Insights (karta Tematy).

Sekcja dodatku na stronie szczegółów agenta lub lokalnego LLM trafia do karty, której dotyczy: Użycie i tokeny, Stan, Tematy (tylko u agentów) lub własna karta z jego nazwą.

Dodatek agenta opisuje swojego agenta jako dane (plik o nazwie `agent.toml`); większość nie przynosi żadnego kodu, a strona dodatku to mówi: *Nie przynosi kodu* albo, dla nielicznych agentów, które czegoś potrzebują (na przykład własnej bazy sesji), *Dane o agencie i trochę kodu na to, czego dane nie potrafią powiedzieć*.

16.6 Nowy agent na twoim komputerze

Gdy instalujesz agenta, którego Maprisma jeszcze nie zna, zagląda do katalogu (Ustawienia › **Szukaj dodatków online**, domyślnie włączone: lista jest czytana z GitHuba co kilka godzin i wysyłane jest tylko zapytanie). Jeśli jakiś dodatek zna tego agenta, mówi o tym stan kolumny (*„Kilo Code jest na tym komputerze, ale nie na mapie”*), a karta pojawia się w **Proponowane dla tego komputera**; kliknięcie go instaluje. Jeśli dodatku jeszcze nie ma, mówią o tym szczegóły agenta, z linkiem, aby o niego poprosić.

16.7 Tworzenie dodatku

Nowy lokalny LLM albo nowy agent potrzebuje tylko dodatku: Maprisma buduje jego kartę, jego pole i linie na mapie, jego szczegóły, stronę Modele i stronę Bezpieczeństwo z tego, co mówi o nim dodatek. Jeśli trochę programujesz, zacznij od szablonów w repozytorium Maprisma (`templates/`, opisanych w `docs/addons.md`): jeden plik dla lokalnego LLM, plik z faktami dla agenta. Wyślij dodatek do `github.com/uBruckhaus/asterism-addons`; po sprawdzeniu i podpisaniu pojawi się dla wszystkich w sekcji **Dostępne**.

ROZDZIAŁ 17

Prywatność i bezpieczeństwo

- **Bez uprawnień administratora dla aplikacji.** Nigdy nie działa jako root i nigdy nie widzi hasła. Agenta dostarczanego jako pakiet systemowy (np. Hermes lub OpenClaw) instaluje menedżer pakietów systemu przez `sudo` w terminalu, który widzisz, po pokazaniu ci polecenia.
- **Bez czytania danych logowania.** Czyta ustawienia agentów, ale nigdy ich plików z danymi logowania. To, czy agent jest zalogowany, rozpoznaje po istnieniu takiego pliku. Wartości kluczy i nagłówków nigdy nie są pokazywane.
- **Bez czytania rozmów.** Liczby użycia są czytane tylko jako liczby i czasy. Jedynym wyjątkiem jest podgląd instancji *W tle* (Terminale…), po twoim kliknięciu i tylko dla tej instancji; nic z tego nie jest zapisywane.
- **Sieć tylko na żądanie** — wyszukiwanie lub instalacja modeli, serwerów MCP czy skilli, test serwera —, plus sprawdzanie aktualizacji co kilka godzin.
- **Sprawdzane pobrania.** Programy są instalowane tylko wtedy, gdy pobrany plik pasuje do sumy opublikowanej przez ich wydanie.
- **Podgląd i Cofnij.** Każda zmiana ustawień innego programu jest najpierw pokazywana, kopiowana zapasowo i można ją cofnąć. Ustawienia, które mogą zawierać klucze, są zapisywane jako czytelne tylko dla ciebie.
- **Ograniczone.** Każdy program działa bez powłoki, a każda odpowiedź jest czytana z limitem rozmiaru i czasu.
- **Funkcje AI tylko wtedy, gdy je wybierzesz.** Wyszukiwania Smart Check działają po kliknięciu, codzienne kontrole AI tylko po włączeniu. Agent dostaje krótkie podsumowanie (nazwy, wersje, modele), nigdy klucze ani wartości ustawień, przekazane tak, że inni użytkownicy komputera nie mogą ich odczytać; jego odpowiedzi są sprawdzane, zanim zostaną pokazane, i nigdy nie są uruchamiane. Zadania działają w terminalu, który widzisz, a agent pyta przed każdym krokiem. Korzystasz z nich na własne ryzyko.

README w każdym repozytorium wymienia w całości, co jest czytane, uruchamiane i zapisywane.

ROZDZIAŁ 18

Rozwiązywanie problemów

Problem	Co zrobić
Mapa jest ściśnięta lub ucięta.	Do 55 % ekranu (poniżej 1150 pikseli) mapa pokazuje kolumny **Agenci** i **Dostawcy**; do 80 % (poniżej 1600 pikseli) dodaje **Modele**; poszerz okno lub włącz więcej kolumn przyciskami nad mapą albo użyj **Dopasuj do okna** (`Ctrl`+`0`).
Na mapie nie ma pola, którego się spodziewam.	Mapa pokazuje tylko to, co jest skonfigurowane: zajrzyj do **Konfiguracji** (wyłączony dodatek, serwer MCP, którego nie używa żaden agent) i do przycisków filtrów kolumn nad mapą.
Brak Uruchom tutaj albo nic nie robi.	Zainstaluj VTE dla GTK 4 (`vte4`) albo użyj **Uruchom w terminalu**.
Linia jest czerwona przerywana.	Kliknij ją, aby zobaczyć wskazówki, napraw to, co mówią, potem **Testuj ponownie**. Zob. Gdy połączenie się nie uda.
Lokalny LLM nie startuje z agentem.	Sprawdź Ustawienia › **Uruchamiaj lokalne LLM razem z agentami** i czy agent jest połączony na mapie. `systemctl --user status ai-agents-map-providers` pokazuje strażnika.
Kolorowe światło od lokalnego LLM do jego modelu się nie pojawia.	Pojawia się tylko wtedy, gdy program spoza mapy generuje z tym LLM, i gaśnie około dziesięciu sekund po jego ostatnim zapytaniu. Sprawdź, czy mapa jest widoczna (szerokie okno) i czy program naprawdę coś wysyła.
Agent odpowiada słabo lub przerywa z lokalnym modelem.	Kontekst może być za mały. Otwórz model w **Modele… › Zainstalowane** i użyj **Ustaw kontekst** z najlepszą wartością dla twojej karty.
Pobieranie się zatrzymało.	Po zerwaniu połączenia wznawia się samo. Po restarcie użyj **Wznów** w pasku u góry.
Chcę cofnąć wcześniejszą zmianę.	Kopie zapasowe są w `~/.local/state/ai-agents-map/backups/` .

DODATEK

Informacje dodatkowe

A.1 Skróty klawiszowe

`F1`	Pomoc
`F9`	F9 zapisuje zrzut ekranu. Obraz okna trafia do `~/Pictures/Maprisma`, domyślnie włączone. W terminalu użyj symbolu aparatu pod nim.
`Ctrl`+`,`	Ustawienia
`F5`	Odśwież (mapę i Konfigurację)
`Ctrl`+`+` / `Ctrl`+`-` / `Ctrl`+`0`	Powiększ mapę, pomniejsz, dopasuj z powrotem do szerokości okna
Kółko myszy	Przewijanie mapy
`Shift`+klik	Dodaj do zaznaczenia lub usuń z niego
`Esc`	Wyczyść zaznaczenie; wróć ze strony
`Backspace`	Wróć ze strony

A.2 Obsługa klawiaturą w podglądzie Flutter

Powyższe skróty działają także w podglądzie Flutter: `F1`, `Ctrl`+`,`, `F5`, `Ctrl`+`+` / `Ctrl`+`-` / `Ctrl`+`0` dla widocznej mapy oraz `Esc` lub `Backspace`, aby wrócić ze strony. W polu tekstowym `Backspace` usuwa, okno dialogowe zamyka się klawiszem `Esc`, a terminal zachowuje oba klawisze dla swojego agenta.

`Tab` przechodzi przez wszystko na ekranie, `Enter` lub `Spacja` tego używa. Na mapie `Enter` na polu je otwiera, a jego przycisk ⋮ otwiera jego działania. W edytorze mapy każde przeciągnięcie ma drogę z klawiatury: **Działania** › **Połącz** pola, potem `Tab` do pola docelowego i `Enter` (`Esc` anuluje); **Przenieś połączenie**, **Przesuń w górę** i **Przesuń w dół** są w tym samym menu. Aplikacja jest sprawdzana pod kątem opisanych elementów sterujących, celów co najmniej 24 × 24 piksele i kontrastu tekstu co najmniej 4,5:1, w jasnym i ciemnym motywie.

A.3 Pliki i foldery

Ścieżka	Zawartość
`~/.config/maprisma/` MAPRISMA `~/.config/omarchy/ai-agents-map/` OMARCHY	Ustawienia, lista serwerów MCP, sprawdzeni agenci, nieudane połączenia.

Ścieżka	Zawartość
`~/.local/state/ai-agents-map/backups/`	Kopie zapasowe każdego pliku zmienionego przez aplikację.
`~/.local/state/ai-agents-map/jobs.json`	Przerwane pobierania i instalacje.
`~/.local/state/ai-agents-map/reorganize.json`	Dziennik ostatnich porządków, dla Cofnij.
`~/.cache/ai-agents-map/`	Wyniki wyszukiwania, listy narzędzi, sprawdzenia aktualizacji, ostatni Smart Check (`smart-check.json`) i aktualizacje MCP z kontroli AI (`mcp-updates.json`).
`~/.local/state/ai-agents-map/ai-help/`	Protokół: jeden plik na każde uruchomienie pomocy AI (ostatnie 200).
`~/.local/state/omarchy-plugin-ai/runs.json`	Wyszukiwania i zadania AI tej aplikacji i innych wtyczek, z ich tokenami: statystyki Smart Check.
`~/.config/systemd/user/ai-agents-map-providers.service`	Strażnik, który uruchamia lokalne LLM razem z ich agentami.

A.4 Słowniczek

Zatwierdzanie Agent pyta, zanim zmieni plik lub uruchomi polecenie (zob. Słowa objaśnione).

Kontekst Ile tokenów tekstu model może mieć jednocześnie na uwadze.

GGUF Format plików modeli używany przez llama.cpp, LM Studio, Ollamę i KoboldCpp.

Prompt injection Ukryte instrukcje na stronie, w pliku lub odpowiedzi narzędzia, które próbują sterować agentem.

Kwantyzacja Zapisanie modelu z mniejszą liczbą bitów na wagę (na przykład *Q4_K_M*, około 4,8 bita), aby był mniejszy, przy niewielkiej utracie jakości.

Mixture of experts Model, w którym dla każdego tokena pracuje tylko część parametrów (*35B-A3B*: 35 miliardów łącznie, 3 miliardy aktywne), co czyni go szybkim jak na swój rozmiar.

Sandbox Odgrodzona przestrzeń, w której działają polecenia agenta, sięgające tylko do folderu projektu i tego, co dozwolone.

Token Kawałek tekstu, mniej więcej trzy czwarte słowa, który modele czytają i piszą.

Usługa użytkownika Program w tle, który systemd uruchamia tylko dla twojego użytkownika, bez uprawnień administratora.